

2212- 03B2 β 03B3 γ 2122TM

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF ELECTRICAL AND ELECTRONICS ENGINEERING

Images/logo.png

GRADUATION THESIS

Project title:

Hardware–Software Co-Design of a Quantized HiFiC GAN Accelerator for Learned Image Compression on FPGA

[NXG1] Loi bien dich tiem an: dong 32 ket thuc bang mot dau \ don le (Accelerator\). Ngoai tabular, mot \ don le tao lenh khong xac dinh / nuot khoang trang, lam vo dong tieu de. Bo dau \ thua hoac thay bang khoang trang.

TRUONG QUOC ANH

anh.tq224430@sis.hust.edu.vn

Class: Electronics – K67

Faculty: Electronics

School: Electrical and Electronics
Engineering

DO VIET DUNG

dung.dv224340@sis.hust.edu.vn

Class: Electronics – K67

Faculty: Electronics

School: Electrical and Electronics
Engineering

Supervisors: Assoc. Prof. Nguyen Duc Minh
Dr. Hoang Phuong Chi

Hanoi, June 13, 2026

PREFACE

In the digital era, the rapid growth of visual data and multimedia content has created an increasing demand for efficient image compression techniques. Traditional compression standards are widely used, but they often struggle to maintain high perceptual quality at very low bitrates. As a result, learned image compression has emerged as a promising approach because it can optimize compression and reconstruction directly from data.

Among learned image compression methods, Generative Adversarial Network (GAN)-based models have shown strong potential for preserving perceptual quality, especially in low-bitrate scenarios. High-Fidelity Image Compression (HiFiC) is a representative framework that applies adversarial learning to improve the visual fidelity of reconstructed images. However, the high computational complexity of GAN-based models creates significant challenges for real-time and embedded deployment.

This thesis investigates a hardware–software co-design approach for accelerating learned image compression based on a quantized HiFiC GAN model. By combining the parallel processing capability of FPGA hardware with flexible software control, the proposed system aims to balance performance, hardware resource usage, and reconstruction quality. This work provides a practical foundation for deploying advanced learned image compression models in real-world systems.

We would like to express our sincere gratitude to Assoc. Prof. Nguyen Duc Minh and Dr. Hoang Phuong Chi for their guidance, support, and valuable feedback throughout the development of this thesis. We also thank the lecturers, researchers, and members of the EDABK laboratory at the School of Electrical and Electronics Engineering, Hanoi University of Science and Technology, for their assistance during system implementation and testing.

Due to limitations in time and experience, this thesis may still contain shortcomings. We welcome feedback from lecturers, reviewers, and colleagues so that this work can be improved in future development.

DECLARATION

We are Truong Quoc Anh and Do Viet Dung. We declare that the thesis entitled “**Hardware–Software Co-Design of a Quantized HiFiC GAN Accelerator for Learned Image Compression on FPGA**” was conducted under the supervision of Assoc. Prof. Nguyen Duc Minh and Dr. Hoang Phuong Chi.

The research, design, simulation, implementation, and evaluation results presented in this thesis are honest and were produced during the execution of the thesis. Any materials, data, or results referenced from other works are properly cited in the References section. We take full responsibility for the content presented in this thesis.

Hanoi, June 13, 2026

Student

(Signature and full name)

Student

(Signature and full name)

Truong Quoc Anh

Do Viet Dung

TABLE OF CONTENTS

LIST OF FIGURES	ii
LIST OF TABLES	iii
ABBREVIATIONS	iv
THESIS SUMMARY	vi
TÓM TẮT	vii
ABSTRACT	viii
CHAPTER 1. INTRODUCTION	1
<i>1.1 Problem Statement</i>	1
<i>1.2 Review of Previous Works</i>	3
<i>1.3 Research Objectives</i>	5
1.3.1 Design Specifications	6
1.3.2 Functional Requirements	7
1.3.3 Non-functional Requirements	8
<i>1.4 Implementation Plan and Timeline</i>	9
<i>1.5 Chapter Conclusion</i>	11
CHAPTER 2. THEORETICAL BACKGROUND	12
<i>2.1 Convolutional Neural Networks</i>	12
2.1.1 Overview of Convolutional Neural Networks	12
2.1.2 Fundamental Layers in CNN Architecture	13
2.1.3 Advantages and Limitations of CNNs for Hardware Acceleration	19
<i>2.2 Generative Adversarial Networks</i>	20
<i>2.3 HiFiC GAN Architecture for Learned Image Compression</i>	21
<i>2.4 Quantization and Numerical Precision</i>	23
<i>2.5 Evaluation Metrics for GAN Accelerator and Image Compression</i>	23
2.5.1 Metrics for GAN Accelerator	23
2.5.2 Metrics for Image Compression	25

2.6 Hardware Platform	26
2.7 High-Level Synthesis	28
2.7.1 Why HLS is Used	28
2.7.2 Scheduling, Binding, and Generated Circuit Structure	29
2.7.3 Pipeline Pragma.....	30
2.7.4 Unroll Pragma.....	31
2.7.5 Array Partition Pragma	32
2.7.6 Bind Operation Pragma.....	33
2.7.7 Interface Pragma	34
2.7.8 Dataflow Pragma.....	37
2.7.9 Summary of HLS Pragma Selection.....	37
2.8 Software Tools for Verification and Synthesis	39
2.8.1 Vivado.....	39
2.8.2 Vitis	39
2.8.3 PetaLinux	40
2.9 Chapter Conclusion	41
CHAPTER 3. IMPLEMENTATION.....	43
3.1 System-Level Architecture Design.....	43
3.1.1 HiFiC GAN Model-Level Data Path.....	44
3.1.2 System Partitioning and PS-PL Interfaces.....	45
3.2 Mixed-Precision Quantization of HiFiC Model.....	47
3.2.1 Quantization Strategy	48
3.2.2 Operator-Level Precision Mapping	48
3.2.3 Quantization Rationale.....	49
3.3 Workload Analysis and Acceleration Target.....	49
3.4 Detailed Internal Block Design	50
3.4.1 Overall Generator Accelerator Architecture.....	50
3.4.2 Fusion Core	51
3.4.3 UpConv Block Core.....	57

3.4.4	Conv77 Core	61
3.4.5	On-Chip Memory Layout	61
3.4.6	Top-Level Kernel Interface	62
3.5	<i>High-Level Synthesis and FPGA Deployment</i>	62
3.6	<i>Chapter Conclusion</i>	64
CHAPTER 4.	EXPERIMENTAL RESULTS	65
4.1	<i>Accuracy Evaluation</i>	65
4.1.1	Verification Environment	65
4.1.2	Functional Verification Results	68
4.1.3	Test Images and Evaluation Metrics	69
4.1.4	Mixed-Precision Quantization Results	70
4.2	<i>Resource Utilization and Power Measurement</i>	72
4.3	<i>Performance Evaluation</i>	74
4.4	<i>Comparison</i>	76
4.5	<i>Chapter Conclusion</i>	77
CHAPTER 5.	CONCLUSION	78

LIST OF FIGURES

Figure 1.1	Annual size of the global datasphere, 2010–2025	2
Figure 1.2	Profiling breakdown of the HiFiC GAN	5
Figure 1.3	Overview of the proposed HiFiC GAN accelerator design and its mapping onto the FPGA platform	6
Figure 2.1	General structure of a convolutional neural network	13
Figure 2.2	Convolution operation.....	14
Figure 2.3	Max pooling.....	16
Figure 2.4	Residual connection	17
Figure 2.5	Four-step computation of a transposed convolution.....	18
Figure 2.6	Scatter and gather views of a 2×2 transposed convolution with stride two	19
Figure 2.7	Overview of the HiFiC GAN-based learned image compres- sion architecture	22
Figure 2.8	HLS compilation, scheduling, binding, and RTL generation flow	28
Figure 2.9	Scheduling and binding example	30
Figure 2.10	Loop pipeline timing example	31
Figure 2.11	Task-level dataflow timing example.....	37
Figure 2.12	PetaLinux project creation, configuration, build, package, and boot flow.....	41
Figure 3.1	Overall design flow of the HiFiC GAN accelerator	43
Figure 3.2	Generator structure with residual blocks and up-convolution blocks	45
Figure 3.3	System-level hardware–software co-design overview of the HiFiC inference pipeline.....	46
Figure 3.4	Mixed-precision quantization architecture applied to convo- lutional layers	47
Figure 3.5	Overall architecture of the HiFiC GAN accelerator	51
Figure 3.6	Computational flow of a residual block in the HiFiC generator	51

Figure 3.7	Two-line buffer structure for streaming convolution.....	52
Figure 3.8	Two-line buffer dataflow and update mechanism	52
Figure 3.9	Sliding window structure for convolution processing	53
Figure 3.10	Parallel window update mechanism with vertical-only move- ment.....	53
Figure 3.11	Reflection padding used in the generator network.....	54
Figure 3.12	Processing Element (PE) architecture for convolution com- putation.....	54
Figure 3.13	Convolution-then-normalization datapath of the Fusion Core, shown for a residual block	56
Figure 3.14	Overlap of the convolution pass and the normalization/acti- vation pass.....	56
Figure 3.15	Block diagram of the Fusion Core	57
Figure 3.16	Network pattern of an up-convolution block (UCB): a 3×3 transposed convolution followed by normalization and ReLU	57
Figure 3.17	Block diagram of the Transpose PE in the UpConv Block Core	59
Figure 3.18	Timing path of the up-convolution engine for the first up- convolution block (UCB_0)	60
Figure 3.19	High-Level Synthesis design flow	63
Figure 4.1	Testing environment: the accelerator under test compared against the HiFiC FP32 reference model	65
Figure 4.2	Verification and deployment environment on the ZCU104 platform.....	66
Figure 4.3	Vivado block design of the proposed accelerator system.....	67
Figure 4.4	Zynq TM UltraScale+ TM MPSoC ZCU104 evaluation board	68
Figure 4.5	PSNR comparison between the Python reference model and the quantized HLS simulation model	70
Figure 4.6	Visual comparison of reconstructed images: FP32 model (left) and FP16 mixed-precision model (right).....	72

LIST OF TABLES

Table 1.1	Survey of representative learned image compression models and the VVC/H.266 baseline	4
Table 1.2	Design specifications of the proposed HiFiC GAN accelerator .	7
Table 1.3	Design Non-functional Requirements and Constraints	8
Table 1.4	Proposed methodologies used in the project	9
Table 1.5	Task planning and scheduling	10
Table 2.1	Comparison between HLS-based design and traditional hand-written RTL design	29
Table 2.2	Summary of HLS pragmas used in the proposed generator accelerator	38
Table 3.1	System-level interfaces used by the generator accelerator	46
Table 3.2	Runtime breakdown of HiFiC generator blocks executed on Intel Core i9 CPU @ 2.2 GHz	49
Table 3.3	Resolution and channel changes through the four up-convolution blocks	58
Table 3.4	Main on-chip buffers of the generator accelerator	61
Table 3.5	AXI master ports of full_generator_top	62
Table 4.1	Relevant ZCU104 components for accelerator evaluation	67
Table 4.2	Block-level functional verification (CSIM) against the FP32 golden reference	69
Table 4.3	Comparison between FP32 and FP16 mixed-precision HiFiC models	71
Table 4.4	Hardware resource utilization of the full HiFiC generator accelerator on the ZCU104	73
Table 4.5	Performance evaluation of the proposed accelerator.....	74
Table 4.6	Compute volume (MAC operations) of the generator blocks ...	75
Table 4.7	Latency breakdown of the generator stages at 300 MHz.....	75
Table 4.8	Per-core throughput and MAC-array efficiency at 300 MHz ...	76

Table 4.9	Performance comparison between the software baseline and the proposed FPGA accelerator	77
-----------	---	----

ABBREVIATIONS

AI	Artificial Intelligence
ANN	Artificial Neural Network
API	Application Programming Interface
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
bpp	Bits Per Pixel
BRAM	Block Random Access Memory
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DDR	Double Data Rate (memory)
DMA	Direct Memory Access
DSP	Digital Signal Processing
FF	Flip-Flop
FP16	16-bit Floating-Point Precision
FP32	32-bit Floating-Point Precision
FPGA	Field-Programmable Gate Array
GAN	Generative Adversarial Network
GOPS	Giga Operations Per Second
HiFiC	High-Fidelity Image Compression
HLS	High-Level Synthesis
IFM	Input Feature Map
LUT	Look-Up Table
MAC	Multiply-Accumulate
MPSoC	Multiprocessor System-on-Chip
MSE	Mean Squared Error
OFM	Output Feature Map
PE	Processing Element
PL	Programmable Logic
PS	Processing System
PSNR	Peak Signal-to-Noise Ratio
RTL	Register-Transfer Level
SoC	System on Chip
SSIM	Structural Similarity Index Measure
URAM	UltraRAM
VCU	Video Codec Unit
VHDL	VHSIC Hardware Description Language

[**NXC1**] Thieu nhieu viet tat duoc dung trong quyen: LPIPS, VVC/H.266, QAT, PTQ, IoT, GMAC/GOP/GOPs, CBI, UCB, FIFO, PIPO, SIMD, ReLU, MS-SSIM, ConvTranspose. Bo sung it nhat LPIPS, VVC, QAT, PTQ, IoT, MS-SSIM. Luu y dung MS-SSIM (Multi-Scale) o Bang 1.1 nhung bang chi co SSIM.

THESIS SUMMARY

This thesis designs and implements an FPGA accelerator for the inference stage of a quantized HiFiC GAN, a Generative Adversarial Network used for learned image compression. The work follows a hardware–software co-design methodology: the computationally dominant residual blocks of the HiFiC generator are accelerated on the Programmable Logic, while model control and auxiliary functions run in software on the Processing System.

The main contributions of the thesis are summarized as follows:

- A post-training mixed-precision quantization of the HiFiC model, with FP32 input/output interfaces and FP16 internal computation, that preserves compression quality: PSNR and SSIM remain almost unchanged compared with the FP32 baseline.
- A full-generator accelerator built with High-Level Synthesis, organized into a Fusion core (initial convolution, nine residual blocks, and the global skip), an up-convolution core, and a final 7×7 convolution core, using line buffers, parallel processing-element clusters, and a rotating accumulator to maximize data reuse and on-chip parallelism. [NXD1] Doi chieu so lieu: Thesis Summary ghi 300 MHz / 122.8 GOPS / 1459x, NHUNG Bang Design Specifications (chuong 1) ghi muc tieu 200 MHz, ≥ 30 GOPS, va Bang Non-functional ghi latency 100x. Day la target vs achieved nen khong sai logic, nhung phai noi ro. Kien nghi them cot 'Achieved' hoac chu thich 'target' trong bang spec.
- A complete implementation on the ZCU104 platform operating at 300 MHz (maximum 308.74 MHz) that reconstructs one 256×256 image in 735.6 ms at a throughput of 122.8 GOPS, about 1459 $\times$ faster than the CPU software baseline.

These results show that FPGA-based acceleration is a practical and effective approach for deploying GAN-based learned image compression, and provide a foundation for more comprehensive hardware deployment in future work.

TÓM TẮT

Những tiến bộ gần đây trong học sâu đã nâng cao đáng kể khả năng nén ảnh bằng cách cho phép các mô hình học tập vượt trội so với các tiêu chuẩn truyền thống trong cả tỷ lệ nén và chất lượng hình ảnh. Đặc biệt, các phương pháp dựa trên Mạng Sinh (GAN) như HiFiC đã chứng minh khả năng mạnh mẽ trong việc bảo toàn các chi tiết hình ảnh tốt nhất ở tỷ lệ bit thấp. Tuy nhiên, độ phức tạp tính toán cao của các mô hình dựa trên GAN gây ra những thách thức lớn cho việc triển khai thời gian thực và các hệ thống nhúng.

Luận văn này trình bày một phương pháp thiết kế kết hợp phần cứng–phần mềm cho nén ảnh dựa trên mô hình HiFiC GAN được lượng tử hóa. Hệ thống được đề xuất nhằm vào tăng tốc mạng HiFiC, trong đó các thành phần tính toán yêu cầu nhiều được tăng tốc trên FPGA, trong khi các chức năng còn lại được thực hiện trên phần mềm. Để cho phép triển khai phần cứng hiệu quả, mạng nơ-ron được lượng tử hóa thành độ chính xác FP16 và được triển khai bằng Tổng hợp cấp cao (HLS), cho phép khai thác hiệu quả tính song song và phát triển thiết kế hợp lý.

Kiến trúc bộ tăng tốc áp dụng mô hình dòng dữ liệu truyền phát với các yếu tố xử lý song song để ánh xạ hiệu quả các phép toán phức tạp vào phần cứng. Thiết kế được triển khai và đánh giá trên nền tảng FPGA ZCU104, chứng minh tần số hoạt động khả thi, sử dụng tài nguyên hợp lý và độ chính xác số tương đương với kết quả tham chiếu phần mềm.

Kết quả thực nghiệm cho thấy rằng khung thiết kế phần cứng–phần mềm đồng thời được đề xuất cung cấp một giải pháp hiệu quả và thực tiễn để tăng tốc các mô hình nén ảnh học tập. Dự án này nhấn mạnh tiềm năng của tăng tốc dựa trên FPGA cho nén ảnh dựa trên GAN nâng cao và đóng vai trò là nền tảng cho các mở rộng trong tương lai hướng tới triển khai phần cứng toàn diện hơn.

ABSTRACT

Recent advances in deep learning have significantly improved image compression by enabling learned models to outperform traditional hand-crafted standards in both compression ratio and perceptual quality. In particular, Generative Adversarial Network (GAN)-based approaches such as High-Fidelity Image Compression (HiFiC) have demonstrated strong capability in preserving fine visual details at low bitrates. However, the high computational complexity of GAN-based models poses major challenges for real-time deployment and embedded systems.

This thesis presents a hardware–software co-design approach for learned image compression based on a quantized HiFiC GAN model. The proposed system targets the inference stage of the HiFiC pipeline, where computationally intensive components are accelerated on FPGA, while the remaining functions are executed in software. To enable efficient hardware implementation, the neural network is quantized to FP16 precision and implemented using High-Level Synthesis (HLS), allowing effective exploitation of parallelism and streamlined design development.

The accelerator architecture adopts a streaming dataflow model with parallel processing elements to efficiently map convolutional operations onto hardware. The design is implemented and evaluated on the ZCU104 FPGA platform, demonstrating feasible operating frequency, acceptable resource utilization, and numerical accuracy comparable to software reference results.

Experimental results indicate that the proposed hardware–software co-design framework provides an effective and practical solution for accelerating learned image compression models. This work highlights the potential of FPGA-based acceleration for advanced GAN-based image compression and serves as a foundation for future extensions toward more comprehensive hardware deployment.

CHAPTER 1. INTRODUCTION

This chapter introduces the research project. It first explains the motivation for accelerating a GAN-based learned image compression model on FPGA and the challenges involved (Problem Statement). It then reviews notable previous works on CNN/GAN hardware accelerators, defines the functional and non-functional requirements that the proposed system must satisfy, and finally presents the implementation plan and the organization of the remainder of this thesis.

1.1 Problem Statement

The Image Data Explosion

The digital world faces an unprecedented explosion in visual data generation. Modern applications across surveillance, medical imaging, remote sensing, and social media platforms continuously produce massive volumes of high-resolution visual content. In 2025 alone, the global data sphere reached 181 zettabytes, with videos accounting for approximately 82% of all internet traffic [?]. This exponential growth creates a critical mismatch with existing storage and processing infrastructure:

[NXD2] MAU THUAN SO LIEU NOI TAI: dong 16 ghi '2025 ... datasphere reached 181 ZB', con danh sach duoi (dong 19) ghi 'projected 175 ZB by 2025'. Hai con so 2025 lech nhau (181 vs 175). Thong nhat 1 con so cho moc 2025, 1 nguon.

- **Rising Storage Costs:** The global datasphere is expanding relentlessly, from 45 ZB in 2019 to a projected 175 ZB by 2025 (Figure 1.1). Storage capacity, however, cannot keep pace: worldwide installed storage was only 6.7 ZB in 2020 – roughly an order of magnitude below the data created that year – less than 2% of newly created data is actually retained, and capacity is expanding more slowly (a 19.2% five-year CAGR) than data generation [?]. Consequently, even as the price per gigabyte of hard-disk and solid-state drives keeps falling, the sheer data volume outpaces these gains, so storage infrastructure remains a significant bottleneck and cost driver for data-intensive applications [?].
- **Edge Processing Demand:** Real-time, high-resolution image compression on embedded IoT and edge devices is essential for surveillance systems, autonomous vehicles, medical devices, and remote monitoring applications. These systems must operate with strict latency, power, and memory constraints.
- **Legacy Compression Limitations:** Traditional standards such as JPEG and JPEG2000 rely on handcrafted transforms and fixed pipelines. They ex-

hibit visible artifacts at high compression ratios and fail to adapt to modern image statistics, limiting their effectiveness for contemporary applications.



Figure 1.1 Annual size of the global datasphere, growing from a few zettabytes in 2010 to a projected 175 ZB by 2025 [?]

The Case for Learned Image Compression

Learned image compression based on deep learning has emerged as a promising solution to address these challenges. Generative Adversarial Network (GAN)-based models, particularly the High-Fidelity Image Compression (HiFiC) framework, have demonstrated state-of-the-art perceptual quality at low bitrates by leveraging adversarial training to generate visually pleasing reconstructions [?]. Unlike traditional schemes, HiFiC preserves fine-grained textures and semantic details by incorporating a generator–discriminator architecture, offering superior rate–distortion performance and adaptability to diverse image content.

However, the deployment of GAN-based compression models in real-world systems faces a critical challenge: substantial computational complexity. The HiFiC inference pipeline consists of deep convolutional networks with multiple residual blocks, high-dimensional feature maps, and intensive arithmetic operations. This makes direct deployment on general-purpose processors (CPUs, GPUs) inefficient for real-time or resource-constrained embedded applications, creating a gap between algorithmic superiority and practical deployability.

Hardware Acceleration on FPGA

Field-Programmable Gate Arrays (FPGAs) provide an attractive platform for addressing this deployment gap. They offer inherent parallelism, fine-grained reconfigurability, and favorable performance–power trade-offs compared to general-purpose processors. Modern heterogeneous FPGA systems enable tight integration between programmable logic and embedded processors (ARM Cortex), facilitating a hardware–software co-design paradigm where computationally intensive kernels

are offloaded to dedicated accelerators while control logic and less demanding tasks execute in software.

Project Motivation

In this work, we present a hardware–software co-design approach for learned image compression based on a quantized HiFiC GAN model targeting embedded deployment. The proposed design accelerates the HiFiC generator pipeline on FPGA using High-Level Synthesis (HLS), with quantization to FP16 mixed-precision to reduce hardware resource requirements while maintaining acceptable numerical accuracy. The resulting system demonstrates how algorithmic innovation (GAN-based compression) and hardware optimization (FPGA acceleration) can be combined to enable practical, real-time image compression on resource-constrained platforms.

1.2 Review of Previous Works

Learned image compression has been an active research area, with GAN-based models such as HiFiC achieving state-of-the-art perceptual quality at low bitrates [?]. On the hardware side, a large body of work has accelerated convolutional neural networks on FPGA by exploiting parallel processing elements, on-chip buffering, and customized dataflow to improve throughput and energy efficiency. However, these efforts have mostly targeted classification or object-detection networks rather than generative image-compression models. A notable recent exception deploys a learned image-compression model on FPGA using knowledge distillation and hybrid quantization to reduce hardware cost [?]; however, it accelerates a non-generative autoencoder codec, whereas this thesis targets a GAN-based perceptual generator.

Table 1.1 surveys representative learned image compression models together with the traditional VVC/H.266 video coding standard. These span traditional coding [?], CNN-based variational autoencoders [?], transformer-based models [?], hybrid CNN–transformer designs [?], and GAN-based perceptual compression [?]. The models optimized purely for rate–distortion (Cheng 2020, STF, TCM) reach higher PSNR and MS-SSIM, whereas HiFiC, which is optimized for perceptual quality, attains the best (lowest) LPIPS at the cost of a lower PSNR. Because perceptual quality at low bitrates is the priority for the target applications, and because the HiFiC generator is built from repeated residual blocks that form a regular, hardware-friendly computational pattern, HiFiC is selected as the model to accelerate in this thesis.

Table 1.1 Survey of representative learned image compression models and the VVC/H.266 baseline

Criterion	VVC/ H.266 [?]	Cheng 2020 [?]	STF [?]	TCM [?]	HiFiC [?]
Year	2020	2020	2022	2023	2020
Type	Traditional	CNN (VAE)	Transformer	CNN + Trans- former	GAN
Backbone	H.266 standard	ResBlock + Attn	Swin Trans- former	TCM Block	Conv + ResBlock
Optimized for	Rate- Distortion	Rate- Distortion	Rate- Distortion	Rate- Distortion	Perceptual
PSNR (dB, ~ 0.4 bpp)	~ 32.0	~ 33.0	$\sim \mathbf{33.8}$	~ 33.5	~ 28.5
MS-SSIM	~ 0.94	~ 0.96	$\sim \mathbf{0.97}$	$\sim \mathbf{0.97}$	~ 0.91
LPIPS	~ 0.40	~ 0.30	~ 0.25	~ 0.23	$\sim \mathbf{0.15}$
Model Params	N/A	$\sim 115\text{M}$	$\sim 599\text{M}$	$\sim \mathbf{50\text{M}}$	$\sim 200\text{M}$
Memory (FP32)	—	~ 440 MB	~ 2.3 GB	~ 190 MB	~ 800 MB

[NXD4] Bang 1.1 (survey) ghi HiFiC PSNR 28.5 dB @ 0.4 bpp. Nhung Bang 4.x (quant results) do PSNR thuc = 19.08 dB @ bpp 0.7961. Chênh lệch (28.5 vs 19.08). Cần giải thích: ảnh test khác tập gốc, model/checkpoint khác, hay cách tính PSNR khác. Nếu không sẽ mâu thuẫn với chính bảng khảo sát.

To the best of our knowledge, the HiFiC GAN model has not previously been implemented on an FPGA. From profiling analysis of the complete HiFiC inference pipeline (Figure 1.2), we observe that the generator module dominates the overall computational cost, accounting for a substantial portion of the total execution time. This motivates the decision to focus hardware acceleration efforts specifically on the HiFiC generator, which offers the greatest potential for performance improvement. Existing learned-compression accelerators do not address the specific computational pattern of the HiFiC generator, which is dominated by repeated residual blocks containing convolution, normalization, activation, and skip connections. This thesis therefore addresses that gap by designing a hardware-software co-design that accelerates the HiFiC generator ResBlock on FPGA. Because no prior FPGA implementation of the same model exists for a direct head-to-head

comparison, the proposed design is instead evaluated against the software (CPU) execution baseline, as presented in Chapter 4.

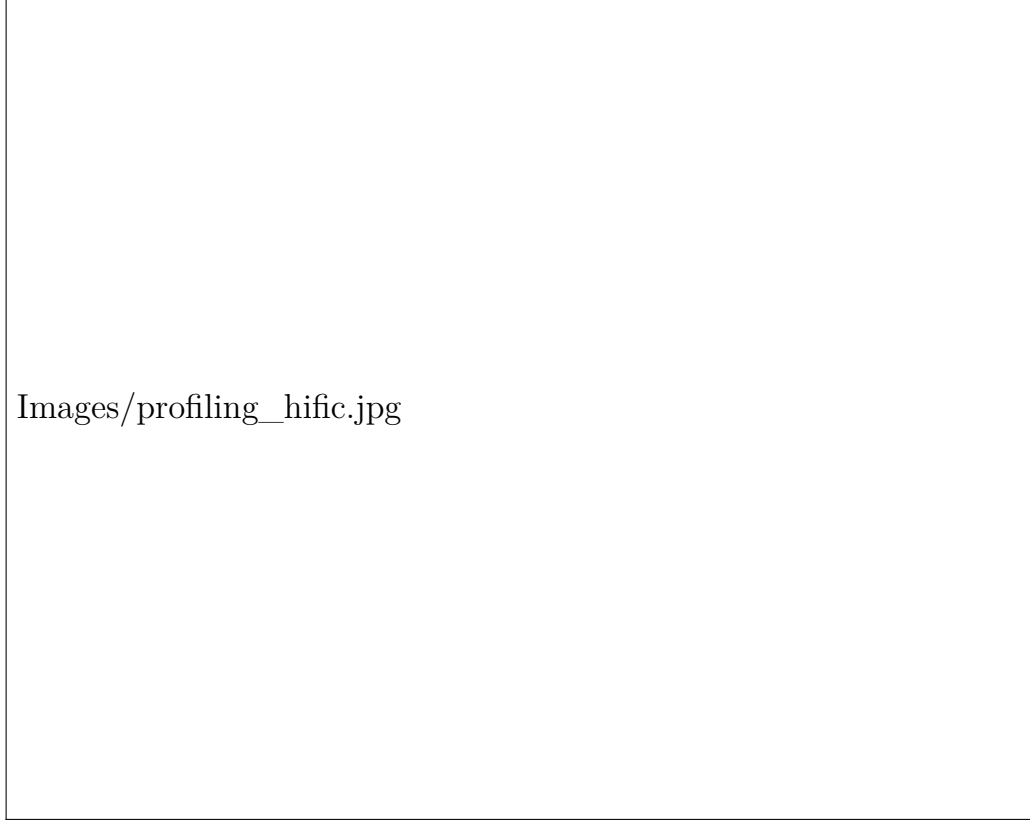


Figure 1.2 Profiling breakdown of the HiFiC GAN

1.3 Research Objectives

The main objective of this thesis is to design and implement an FPGA-based accelerator for the quantized HiFiC GAN inference pipeline using a hardware–software co-design methodology. The specific contributions of the project are summarized as follows:

- A hardware–software co-design framework for HiFiC GAN inference targeting learned image compression applications.
- An FPGA-based accelerator architecture optimized for quantized GAN workloads using streaming dataflow and parallel processing elements.
- A practical implementation and evaluation on the ZCU104 FPGA platform, demonstrating feasible performance, resource efficiency, and numerical accuracy.

1.3.1 Design Specifications

The high-level design specifications of the proposed accelerator are summarized in Table 1.2. They define the target model, hardware platform, tool flow, and the main performance and precision targets that the implementation in Chapter 3 must satisfy.

Figure 1.3 gives an overview of the design. The HiFiC generator transforms the hyper-latent representation into the reconstructed $256 \times 256 \times 3$ RGB image through a convolution input block, the residual blocks, the up-convolution blocks, and a final output convolution. These stages are mapped onto the three cores of the GAN accelerator (Fusion Core, UpConv Block Core, and Conv77 Core), which exchange data over an AXI4 interconnect and are integrated with the host processor and external DRAM on the FPGA platform.

Images/Design_spec_ver2.png

Figure 1.3 Overview of the proposed HiFiC GAN accelerator design and its mapping onto the FPGA platform

Table 1.2 Design specifications of the proposed HiFiC GAN accelerator

Specification	Target
Core design	Quantized HiFiC GAN generator accelerator (Fusion, UpConv, and Conv77 cores)
Target model	HiFiC GAN inference for learned image compression and reconstruction
Input / output	Input: hyper-latent tensor ($16 \times 16 \times 960$); output: reconstructed RGB image ($256 \times 256 \times 3$)
Target device	Xilinx Zynq UltraScale+ MPSoC ZCU104
Development flow	Vitis HLS, Vivado, and Vitis (embedded software)
Operating frequency	200 MHz
Throughput	≥ 30 GOPS (Giga Operations Per Second)
Quantization	Mixed-precision FP16 (FP32 reference baseline)

[NXD3] Bang Design Specifications: Operating frequency = 200 MHz, Throughput ≥ 30 GOPS. Ket qua thuc (chuong 4) la 300 MHz / 122.8 GOPS. Neu la 'target' ban dau thi ghi ro tu 'target/minimum'; neu da chot 300 MHz thi cap nhat. Hien so chuong 1 va chuong 4 vênh nhau.

1.3.2 Functional Requirements

The proposed system must satisfy the following functional requirements:

- **Accelerator Input and Output:** Take the hyper-latent representation ($16 \times 16 \times 960$) produced by the compression front-end as input and generate the reconstructed RGB image ($256 \times 256 \times 3$) as output.
- **Accelerator Kernel:** Implement a hardware accelerator for the complete HiFiC generator (decoder), composed of the Fusion Core (initial convolution block, nine residual blocks, and the global skip addition), the UpConv Block Core (four transposed-convolution up-sampling blocks), and the Conv77 Core (final 7×7 output convolution).
- **Model Support:** Support the quantized HiFiC GAN inference pipeline with mixed-precision arithmetic (FP16 for computations, FP32 for reference baseline).
- **Hardware–Software Interface:** Implement PS–PL communication through AXI4-Lite (control signals) and AXI Master (data movement) to enable seamless integration with the ARM Cortex-A53 processor on the ZCU104 platform.

- **Functional Correctness:** Guarantee that hardware-accelerated compression produces numerically equivalent results compared to the software (CPU) reference implementation within acceptable tolerance bounds.
- **Compression Pipeline:** Support the complete image compression and reconstruction flow, from input image to final decompressed output with maintained visual quality.

1.3.3 Non-functional Requirements

Beyond the headline targets listed in the design specifications (Table 1.2), the implementation must satisfy the following non-functional requirements:

Table 1.3 Design Non-functional Requirements and Constraints

Requirement Category	Target Specification
FPGA LUT	
FPGA Flip-Flop	
FPGA BRAM	
FPGA URAM	
FPGA DSP	
Memory Bandwidth	Efficient utilization of AXI bandwidth for data movement between DDR and accelerator
Accuracy Preservation	Maintain image quality metrics (PSNR, MS-SSIM, LPIPS) within acceptable tolerance compared to FP32 reference
Latency	Achieve 100x speedup over the Intel Core i9 CPU software baseline
Power Efficiency	Demonstrate energy efficiency improvements over a pure software (CPU) implementation

[NXF1] Bang Non-functional Requirements: cac hang FPGA LUT/FF/BRAM/URAM/DSP de TRONG cot Target. Bang 'yeu cau/rang buoc' khong co nguong dinh luong thi khongkiem chung duoc o chuong 4. Hoac dien nguong muc tieu, hoac bo cac hang trong.

[NXF2] 'Latency: Achieve 100x speedup over Intel Core i9' — chuong 4 dat 1459x. 100x vs 1459x chenh 1 bac do lon; ra soat con so muc tieu 100x co dung khong, keo nguoi phan bien thac mac do do chinh xac phep do.

1.4 Implementation Plan and Timeline

To meet the design specifications and requirements defined above, this project adopts a set of hardware-oriented methodologies, summarized in Table 1.4. These methodologies follow directly from the streaming, convolution-heavy nature of the HiFiC generator and from the resource constraints of the ZCU104 platform.

Table 1.4 Proposed methodologies used in the project

No.	Methodology	Impact
1	Hardware–software co-design partitioning	Keeps the irregular encoder, hyperprior, and entropy stages in software and offloads the compute-bound generator to the FPGA, removing the dominant runtime bottleneck
2	High-Level Synthesis (Vitis HLS) design flow	Generates and iterates RTL from a C/C++ reference, shortening the design cycle
3	Mixed-precision FP16 quantization	Reduces memory footprint and on-chip resource usage while preserving reconstruction quality
4	Streaming line-buffer and sliding-window convolution	Converts repeated external-memory traffic into on-chip data reuse for high-throughput convolution
5	Two-pass convolution–normalization with a rotating accumulator	Overlaps the convolution and normalization passes and sustains an initiation interval of one, maximizing MAC utilization
6	Single shared, reusable compute engine per core	Reuses one hardware instance across the repeated residual blocks and stages, fitting the design within the tight on-chip resource budget

The overall work is divided into a sequence of phases, each with a defined deliverable, as presented in Table 1.5.

Table 1.5 Task planning and scheduling

No.	Task	Person in charge	Period	Result
1	Survey learned image compression models and prior CNN/GAN FPGA accelerators	Truong Quoc Anh	2 weeks	Define the design specifications and evaluation metrics
2	Analyze the HiFiC generator and profile its workload	Truong Quoc Anh	1 week	Identify the residual blocks as the acceleration target and decide the HW/SW partition
3	Build the C/C++ reference model and testbench from the pre-trained HiFiC model	Do Viet Dung	2 weeks	Establish a golden reference for functional verification
4	Design and implement the HLS accelerator cores (Fusion, UpConv, Conv77)	Truong Quoc Anh, Do Viet Dung	4 weeks	A functional generator accelerator described in synthesizable C/C++
5	Apply mixed-precision quantization and HLS optimizations (pipelining, array partitioning, ping-pong buffering)	Truong Quoc Anh, Do Viet Dung	3 weeks	Meet the throughput and resource targets
6	Verify by C-simulation and C/RTL co-simulation, then integrate on the ZCU104 (Vivado, Vitis)	Truong Quoc Anh, Do Viet Dung	3 weeks	A hardware-verified accelerator running on the board
7	Evaluate accuracy, resource utilization, and performance against the CPU baseline; write the thesis and slides	Do Viet Dung	3 weeks	Comprehensive results and the graduation report

1.5 Chapter Conclusion

This chapter presented the motivation, objectives, and scope of the thesis. The remainder of this thesis is organized as follows:

- **Chapter 2 – Theoretical Background:** presents the background of CNNs, GANs and the HiFiC architecture, evaluation metrics, the FPGA platform, the High-Level Synthesis methodology, and the software tools used in this thesis.
- **Chapter 3 – Implementation:** describes the system-level architecture, the mixed-precision quantization strategy, the workload analysis, and the internal accelerator blocks of the proposed full generator accelerator.
- **Chapter 4 – Experimental Results:** presents the verification procedure, accuracy and quantization results, FPGA resource utilization, performance evaluation, and comparison results.
- **Chapter 5 – Conclusion:** summarizes the main achievements of the project and discusses potential directions for future work.

At the end of this thesis, the consulted literature is presented in the References section.

CHAPTER 2. THEORETICAL BACKGROUND

Chapter 2 presents the theoretical background required for the design and implementation of the proposed quantized HiFiC GAN accelerator. The chapter first reviews convolutional neural networks (CNNs) and their fundamental layers, since the generator network in learned image compression is dominated by convolution, normalization, activation, residual, and up-sampling operations. It then introduces Generative Adversarial Networks (GANs) and the HiFiC architecture used for learned image compression, followed by the mixed-precision quantization concept that underpins the proposed accelerator. Next, the chapter presents the evaluation metrics for both hardware efficiency and image-compression quality, summarizes the FPGA platform used for hardware acceleration, and introduces High-Level Synthesis (HLS), explaining why HLS is suitable for this thesis and discussing the HLS pragmas used to transform the ResBlock algorithm into an efficient FPGA circuit. Finally, it reviews the software tools used in the design flow.

2.1 Convolutional Neural Networks

2.1.1 *Overview of Convolutional Neural Networks*

Convolutional Neural Networks (CNNs) are a major class of deep learning models designed for processing data with spatial structure [?]. They are especially effective for two-dimensional visual data such as images, where neighboring pixels usually have strong correlations. Instead of treating every input element independently, CNNs exploit local spatial relationships through convolutional filters. This property makes CNNs suitable for many computer vision tasks, including image classification, object detection, image segmentation, image restoration, and learned image compression.

The operation of a CNN can be interpreted as a hierarchical feature extraction process. In the first layers, the network usually learns simple visual patterns such as edges, corners, color transitions, and local textures. As the data propagates through deeper layers, these low-level patterns are gradually combined into more abstract representations, such as object parts, semantic regions, or compact latent features. In learned image compression systems, this hierarchical representation is important because the encoder network must transform the input image into a compressed latent representation while preserving perceptually meaningful information for the decoder.

A general CNN architecture is shown in Figure 2.1. The input image is

processed by a sequence of convolutional blocks. Each block commonly contains a convolutional layer, a nonlinear activation function, and sometimes additional operations such as normalization or pooling. For image compression and generative models, the network may also include down-sampling layers in the encoder to reduce spatial resolution and produce compact latent features.



Figure 2.1 General structure of a convolutional neural network [?]

Compared with a fully connected neural network, a CNN requires fewer parameters for image-related tasks because it uses local connectivity and weight sharing. In a fully connected layer, each output neuron is connected to all input elements, which leads to a very large number of parameters for high-resolution images. In contrast, a convolutional layer uses a small kernel that slides across the input feature map. The same kernel weights are reused at different spatial locations, enabling the network to detect similar features regardless of their positions in the image. This mechanism reduces memory requirements and improves generalization.

2.1.2 Fundamental Layers in CNN Architecture

A CNN is constructed from several types of layers. Each layer has a specific role in transforming the input data into useful feature representations. The most common components are convolutional layers, activation layers, pooling or down-sampling layers, and fully connected layers. In modern image compression and generative architectures, fully connected layers are often avoided or used only in small auxiliary networks because convolutional layers are more efficient for preserving spatial structure.

2.1.2.1 Convolutional Layer

The convolutional layer is the main computational component of a CNN. Its purpose is to extract local features from the input by applying a set of learnable kernels. For an input feature map, each kernel scans over local receptive fields and produces one output feature map. When multiple kernels are used, the layer generates multiple output channels, where each channel corresponds to a different learned feature detector.

The convolution operation is illustrated in Figure 2.2. At each spatial position, the kernel values are multiplied by the corresponding input values, and the products are accumulated to form one output value. This multiply-accumulate pattern dominates the computational cost of CNN inference and is therefore a key target for hardware acceleration.

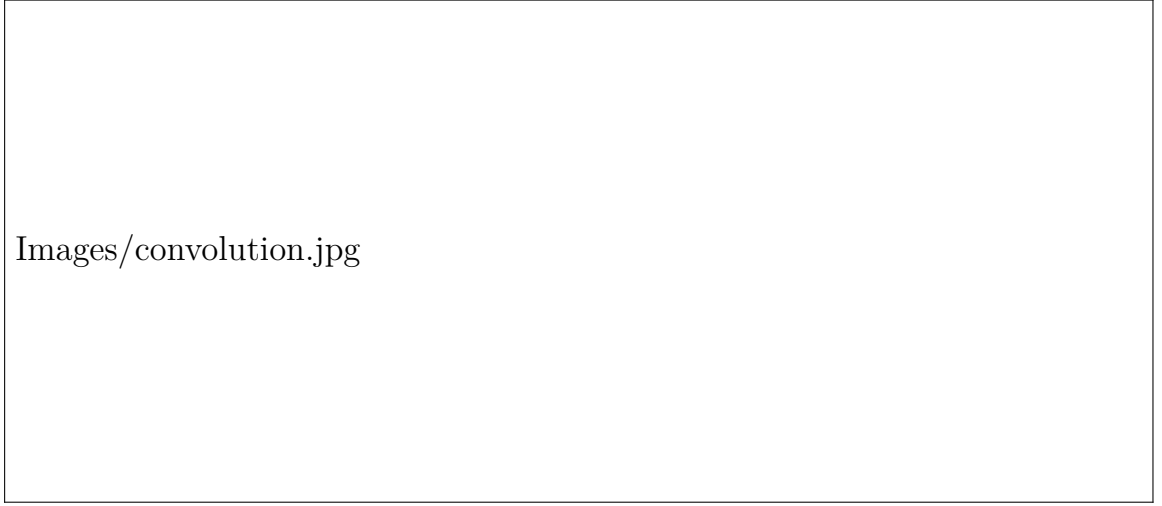


Figure 2.2 Convolution operation

For a convolutional layer, the spatial size of the output feature map depends on the input size, kernel size, stride, and padding. If the input height and width are denoted by H and W , the kernel height and width by K_h and K_w , the padding by P , and the stride by S , then the output height H_{out} and output width W_{out} can be calculated as

$$H_{out} = \left\lfloor \frac{H - K_h + 2P}{S} \right\rfloor + 1, \quad (2.1)$$

$$W_{out} = \left\lfloor \frac{W - K_w + 2P}{S} \right\rfloor + 1. \quad (2.2)$$

Stride controls how far the kernel moves after each computation. A larger stride reduces the output resolution and decreases the number of operations. Padding adds values, usually zeros, around the input boundary so that border information

can be included in the convolution. Padding is also used to preserve the spatial size of feature maps when necessary.

2.1.2.2 *Activation Function*

A convolution operation is linear. Therefore, activation functions are inserted after convolutional layers to introduce nonlinearity into the network. Without nonlinear activation, a deep stack of convolutional layers would still be equivalent to a linear transformation and would have limited representational power. Common activation functions include Rectified Linear Unit (ReLU), sigmoid, hyperbolic tangent (tanh), and Swish.

ReLU is widely used because of its simple hardware-friendly computation and its ability to reduce the vanishing gradient problem. It is defined as

$$f(x) = \max(0, x). \quad (2.3)$$

For more complex neural networks, other activation functions may be selected to improve accuracy or perceptual quality. In hardware implementation, the activation function must be considered carefully because nonlinear functions such as sigmoid or Swish are more expensive to implement than ReLU. For a quantized accelerator, this issue becomes even more important because the activation function must be compatible with fixed-point arithmetic and efficient lookup or approximation methods.

2.1.2.3 *Pooling and Down-Sampling Layers*

Pooling layers reduce the spatial size of feature maps, which lowers computational complexity and decreases memory usage. In particular, max pooling selects the largest activation within each local window, producing a down-sampled feature map while preserving the most salient features from the input region. Figure 2.3 illustrates this process: each pooling window is replaced by its maximum value, so the output feature map retains strong responses while its spatial dimensions are reduced.

Average pooling is another common method, but in image-compression and generative models, max pooling is often preferred when the goal is to preserve strong feature responses. In many modern CNN-based compression and generation models, down-sampling may also be performed by strided convolution instead of a separate pooling layer. Strided convolution has learnable parameters, so it can reduce resolution while also extracting features. This approach is useful in encoder networks, where the image must be transformed into a lower-resolution latent representation.

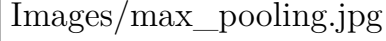


Figure 2.3 Max pooling

2.1.2.4 Normalization Layer

Normalization layers rescale intermediate feature maps to a consistent statistical range, which stabilizes and accelerates training. Common variants include batch normalization, instance normalization, and channel (layer) normalization, which differ in the dimensions over which the mean and variance are computed. For a feature value x , normalization can be expressed as

$$\hat{x} = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad (2.4)$$

where μ and σ^2 are the mean and variance computed over the chosen dimensions, ϵ is a small constant for numerical stability, and γ and β are learnable scale and shift parameters. The HiFiC generator applies channel normalization inside its residual blocks, so each residual block contains convolution, normalization, and activation. In a quantized hardware implementation, normalization must be handled carefully because its division and square-root operations are more expensive than multiply-accumulate and are sensitive to reduced precision.

2.1.2.5 Residual Connections

As illustrated in Figure 2.4, a residual connection adds the input of a block directly to its output through a skip path, so that the convolutional path only has to learn a residual mapping $F(x)$ instead of a full transformation:

$$y = x + F(x), \quad (2.5)$$

where x is the input feature map and $F(x)$ is the transformation produced by the convolutional path. Residual connections improve gradient flow and feature reuse, which makes deep networks easier to train.

The building block that implements this pattern is called a residual block (ResBlock). In the HiFiC generator, the ResBlock combines convolution, normalization, activation, and a skip connection, and is repeated many times. Because



Images/residual_connection.png

Figure 2.4 Residual connection

these repeated ResBlocks dominate the inference computation, they are the main target for hardware acceleration in this thesis.

2.1.2.6 Transposed Convolution and Up-Sampling

While the encoder reduces spatial resolution, the generator must increase resolution to reconstruct the output image. This is achieved by up-sampling operations, most commonly transposed convolution (also called deconvolution or fractionally-strided convolution) [?]. Whereas a normal convolution performs a many-to-one mapping that aggregates a local input window into a single output value, transposed convolution performs the inverse one-to-many mapping: each input value is multiplied by the entire kernel and its contribution is spread across a neighbourhood of output positions, thereby expanding the feature map. It provides a *learnable* up-sampling, in contrast to fixed methods such as nearest-neighbor or bilinear interpolation.

For an input of size i , kernel size k , stride s , and padding p , the output size of a transposed convolution is

$$o = (i - 1) s + k - 2p. \quad (2.6)$$

Unlike the normal convolution formula, no floor operation appears because the stride multiplies the spacing between projected input samples instead of dividing the input size; a stride $s > 1$ therefore *increases* the output resolution. In the HiFiC generator, several up-convolution blocks each apply a 3×3 transposed convolution with stride two, progressively restoring the spatial resolution of the latent representation back to the original image size.

In practice, deep-learning frameworks compute a transposed convolution by reducing it to an ordinary convolution, as illustrated in Figure 2.5. Starting from

the stride s and padding p , the input is first expanded by inserting $z = s - 1$ zeros between adjacent rows and columns; a border of $p' = k - p - 1$ zeros is then added around the expanded input; finally a standard convolution with unit stride is slid over the result to produce the up-sampled output. This zero-insertion procedure is what allows a network to learn the up-sampling weights while reusing a conventional convolution operator.

Images/transposed_conv_steps.png

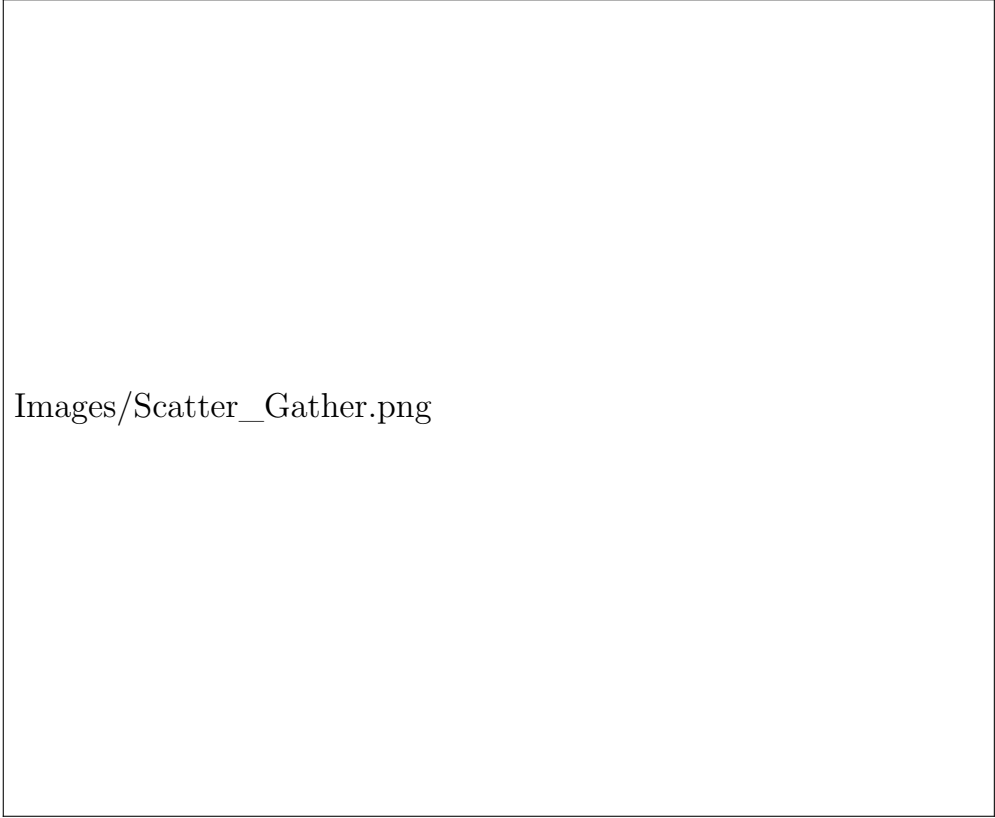
Figure 2.5 Four-step computation of a transposed convolution [?]

2.1.2.7 Scatter and Gather Views of Transposed Convolution

Transposed convolution can be computed in two mathematically equivalent ways, illustrated in Figure 2.6.

In the **scatter** view (top of Figure 2.6), the operation follows its natural definition. Each input pixel is multiplied by every kernel tap, producing a weighted copy of the kernel that is placed (scattered) onto the output grid at a position determined by the stride. Because the strided placements of neighbouring input pixels overlap, output positions that receive contributions from several input pixels are summed. A single output pixel is therefore written several times and is only complete after all input pixels have scattered their contributions, which requires an accumulation buffer over the output.

In the **gather** view (bottom of Figure 2.6), the same result is obtained by reducing the problem to a standard convolution: zeros are inserted between the input samples (and around the border) according to the stride, the kernel is flipped (180° rotation), and an ordinary stride-one convolution is then applied to the zero-expanded input. Each output pixel now gathers a fixed local window, exactly as in a normal convolution. This view is convenient for analysis and for reusing a standard convolution engine, but the inserted zeros lead to wasted multiplications.



Images/Scatter_Gather.png

Figure 2.6 Scatter and gather views of a 2×2 transposed convolution with stride two

The choice between the scatter and gather formulations directly shapes the hardware design of the up-convolution accelerator presented in Chapter 3: the scatter formulation avoids the wasted multiplications of the zero-inserted gather formulation, but in exchange requires a row-wide accumulation buffer because each output pixel is updated several times.

2.1.3 Advantages and Limitations of CNNs for Hardware Acceleration

CNNs have several advantages for hardware implementation. First, convolution operations contain a high degree of data reuse. The same weights are applied to many spatial positions, and neighboring output pixels often share overlapping input windows. This reuse can be exploited through on-chip buffers and parallel processing elements. Second, CNN layers are highly parallel because many output pixels and output channels can be computed independently. This parallelism is suitable for FPGA and ASIC accelerators. Third, CNN models can be optimized using quantization, pruning, and layer fusion to reduce computation and memory traffic.

However, CNNs also create several hardware challenges. Deep CNNs require a

large number of multiply-accumulate operations, which may exceed the capability of general-purpose processors when real-time performance is required. In addition, feature maps and model weights can occupy significant memory, leading to high bandwidth demand between external memory and the accelerator. Some layers, such as normalization and complex activation functions, may have irregular access patterns or nonlinear operations that are less straightforward to accelerate.

For the quantized HiFiC GAN accelerator in this thesis, these advantages and limitations motivate a hardware-software co-design approach. The software side is responsible for model preparation, quantization, parameter management, and system control, while the hardware side accelerates the dominant CNN computations. By designing both sides together, the system can better satisfy the requirements of learned image compression, including high throughput, reduced resource usage, and acceptable reconstructed image quality.

2.2 Generative Adversarial Networks

[NXF3] Dau [?]at SAU dau cham, tach khoi cau (vd '... visually plausible. [?]). IEEE style nen dat truoc dau cham cuoi cau. Lap lai o dong 155,157,159,163,165... Generative Adversarial Networks (GANs) are a class of generative models designed to learn the distribution of real data and synthesize new samples that appear realistic. A typical GAN consists of two neural networks trained in competition: a generator and a discriminator. The generator attempts to produce data samples that resemble the training distribution, while the discriminator attempts to distinguish generated samples from real samples. Through this adversarial training process, the generator gradually improves its ability to create outputs that are visually plausible. [?]

In image-related applications, GANs are particularly useful because they can encourage outputs to match the perceptual characteristics of natural images rather than only minimizing pixel-wise error. This is important because traditional distortion metrics, such as Mean Squared Error (MSE), often favor overly smooth reconstructions when the compression ratio is high. Although such reconstructions may achieve acceptable numerical distortion, they can lose fine textures and perceptually important details. GAN-based objectives address this limitation by comparing the distribution of reconstructed images with the distribution of real images, thereby improving perceptual realism. [?, ?]

The use of GANs in learned image compression is motivated by the rate-distortion-perception trade-off. At a fixed bitrate, optimizing only distortion does not necessarily lead to the best perceptual quality. Conversely, improving per-

ceptual quality may increase the measured distortion between the original and reconstructed images. Therefore, GAN-based compression methods introduce an adversarial loss to guide the reconstruction toward the natural image manifold. This makes the reconstructed image visually more convincing, especially at low bitrates where classical codecs may produce blocking or banding artifacts and distortion-optimized neural codecs may produce blur. [?, ?]

However, GAN-based compression models also introduce implementation challenges. The generator network usually contains many convolutional and residual blocks, which require substantial multiply-accumulate operations and memory bandwidth during inference. In addition, the numerical behavior of a GAN-based decoder can be sensitive to quantization because small changes in intermediate feature maps may affect perceptual details in the reconstructed image. These characteristics motivate a hardware-software co-design approach, where software handles model preparation and system control while hardware accelerates the most computationally intensive neural network blocks. [?, ?]

2.3 HiFiC GAN Architecture for Learned Image Compression

High-Fidelity Image Compression (HiFiC) is a GAN-based learned image compression framework that combines neural compression with adversarial learning to improve perceptual reconstruction quality. Instead of optimizing only for pixel-wise similarity, HiFiC aims to reconstruct images that are visually close to the input while maintaining practical bitrates. The original HiFiC study showed that GAN-based compression can produce high-fidelity reconstructions and can be evaluated using both perceptual metrics and human preference studies. [?]

The overall HiFiC architecture used in this thesis can be divided into three main components: an encoder network, a hyperprior network, and a generator network, as shown in Figure 2.7. The encoder transforms the input RGB image into a compact latent representation. This latent representation contains the information required for reconstruction while removing spatial redundancy from the input image. The hyperprior network models the statistical distribution of the latent variables and provides side information for more efficient entropy coding. Finally, the generator reconstructs the image from the compressed latent representation. [?, ?]

The encoder is implemented as a sequence of convolutional blocks. Each block extracts hierarchical image features while reducing redundancy in the spatial domain. Unlike a conventional autoencoder trained only with a pixel-wise loss, the HiFiC encoder is optimized as part of a perceptual compression system. As a result,



Figure 2.7 Overview of the HiFiC GAN-based learned image compression architecture

the latent representation is expected to preserve visually meaningful information that supports high-quality reconstruction at the decoder side. [?]

The hyperprior network improves compression efficiency by learning side information about the latent representation. This side information, often called hyper-latent variables, helps estimate the entropy model used for coding the compressed representation. A more accurate entropy model allows the system to assign shorter codes to more likely latent symbols, which reduces bitrate while maintaining reconstruction quality. [?]

The generator network is responsible for reconstructing the output image from the latent and hyper-latent information. In the HiFiC generator design, the generator contains an initial convolutional block, multiple residual blocks, reconstruction blocks, and a final convolutional block. The residual blocks include convolution, normalization, activation functions, and skip connections. These skip connections improve feature reuse and help stabilize the reconstruction process. Because residual blocks contain repeated convolution-heavy operations, they dominate the computational workload and become a natural target for hardware acceleration. [?, ?]

For the hardware-software co-design in this thesis, the generator is selected as the primary acceleration target because it contains most of the inference-time computation. The remaining components can be executed in software to preserve flexibility in model control, data formatting, and system integration. This partitioning provides a practical balance between hardware efficiency and software programmability, which is important for deploying a quantized HiFiC model on FPGA-based embedded platforms. [?, ?]

2.4 Quantization and Numerical Precision

Neural networks are typically trained using 32-bit floating-point (FP32) arithmetic, which provides high numerical precision but is expensive in terms of memory footprint, memory bandwidth, and arithmetic resource usage. Quantization reduces the numerical precision of weights and activations to a lower-bit representation so that the model requires less storage and can be executed with smaller, faster arithmetic units. This is particularly important for FPGA deployment, where on-chip memory and DSP resources are limited.

Quantization methods are generally divided into two categories. Quantization-aware training (QAT) simulates reduced precision during training so that the model can adapt to quantization noise, whereas post-training quantization (PTQ) converts an already trained FP32 model to lower precision without retraining. PTQ is faster to deploy and is the approach adopted in this thesis.

The numerical format also matters. Integer formats such as INT8 offer the smallest footprint but have a limited dynamic range, which can be problematic for GAN-based decoders where small numerical errors may produce visible artifacts. Half-precision floating-point (FP16) keeps a floating-point exponent, so it preserves a wider dynamic range than integer formats while still halving the storage and increasing the arithmetic density compared with FP32. For this reason, the proposed accelerator adopts a mixed-precision scheme in which the input and output interfaces remain in FP32 for compatibility, while the internal computation of the generator is performed in FP16. The detailed quantization strategy applied to the HiFiC model is presented in Chapter 3.

2.5 Evaluation Metrics for GAN Accelerator and Image Compression

Evaluation metrics are required to assess both the hardware efficiency of the proposed GAN accelerator and the reconstruction quality of the learned image compression model. Since this thesis focuses on a quantized HiFiC GAN accelerator, the evaluation should consider two complementary aspects. The first aspect measures how efficiently the accelerator executes the neural network workload on hardware. The second aspect measures how close the reconstructed image is to the original image after compression and decompression.

2.5.1 Metrics for GAN Accelerator

The performance of a GAN accelerator can be evaluated using several hardware-oriented metrics. These metrics are adapted from common CNN accelerator evaluation criteria because the generator network in HiFiC is mainly composed of

convolutional and residual blocks. Therefore, latency, throughput, inference speed, power consumption, power efficiency, and FPGA resource utilization are important indicators for analyzing the efficiency of the proposed design. [?]

[NXB1] Cong thuc $Latency = n_T \times T_{vanT} / F$ dung.Ca ndinh nghia $r_{onT}(sochuky) cho TOAN b$ stageochuong4.Latency measuresthe timerequiredbytheaccelerator to processoneinputimageandprodtimeornear-real-timeimagecompression systems, lowerlatencyisdesirablebecauseitreducesthewa clock cycles and the clock period is T , the latency can be written as

$$Latency = n_T \times T. \quad (2.7)$$

Since the clock period is the inverse of the operating frequency F , the latency can also be expressed as

$$Latency = \frac{n_T}{F}. \quad (2.8)$$

Throughput represents the amount of computation completed per second. For neural network accelerators, it is commonly expressed in Giga Operations Per Second (GOPS). A higher throughput indicates that the hardware can process more arithmetic operations in a fixed time interval. If the computational workload of the model is denoted by *Workload* in GOPs, the throughput is calculated as

$$Throughput (GOPS) = \frac{Workload (GOPS)}{Latency (s)}. \quad (2.9)$$

Inference speed describes how many images or frames can be processed per second. It is usually measured in frames per second (fps). For an image compression accelerator, this metric indicates how many reconstructed images can be generated in one second. The inference speed is the inverse of latency:

$$Inference Speed (fps) = \frac{1}{Latency (s)}. \quad (2.10)$$

Power consumption is another important metric, especially for FPGA-based embedded systems. It indicates the amount of electrical power consumed by the accelerator during execution. The total power may include the power used for computation, on-chip memory access, external memory transfer, control logic, and input/output interfaces. Lower power consumption is beneficial for portable or energy-constrained systems.

Power efficiency evaluates how much computational performance is achieved for each watt of power. It is calculated as the ratio between throughput and power consumption:

$$Power Efficiency (GOPS/W) = \frac{Throughput (GOPS)}{Power Consumption (W)}. \quad (2.11)$$

A higher value means that the accelerator can deliver more computation with less energy, which is desirable for practical deployment.

Resource utilization is used to analyze how much of the FPGA fabric is occupied by the accelerator. The main resource metrics include logic utilization, DSP utilization, and BRAM utilization. Logic utilization reflects the amount of configurable logic used for control and data-path implementation. DSP utilization shows how many dedicated arithmetic blocks are used for multiply-accumulate operations. BRAM utilization indicates the amount of on-chip memory used for storing input feature maps, intermediate feature maps, weights, and temporary buffers. These metrics help determine whether the accelerator is compact, scalable, and suitable for the target FPGA platform.

2.5.2 Metrics for Image Compression

In addition to hardware metrics, image compression quality must be evaluated by comparing the reconstructed image with the original image. Two commonly used objective metrics are Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM). PSNR measures pixel-wise reconstruction fidelity, while SSIM evaluates structural similarity from a perceptual perspective. [?, ?]

PSNR is derived from the Mean Squared Error (MSE) between the original image and the reconstructed image. For an image with height H , width W , and C color channels, the MSE is defined as

$$MSE = \frac{1}{HWC} \sum_{i=1}^H \sum_{j=1}^W \sum_{k=1}^C \left(I(i, j, k) - \hat{I}(i, j, k) \right)^2, \quad (2.12)$$

where I is the original image and $\hat{I}$ is the reconstructed image. PSNR is then computed as

$$PSNR = 10 \log_{10} \left(\frac{MAX_I^2}{MSE} \right), \quad (2.13)$$

where MAX_I is the maximum possible pixel value. For 8-bit images, $MAX_I = 255$. A higher PSNR generally indicates lower pixel-level distortion and better reconstruction fidelity.

SSIM measures the similarity between two images by comparing luminance, contrast, and structural information. For two image patches x and y , SSIM is commonly expressed as

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (2.14)$$

where μ_x and μ_y are the mean intensities, σ_x^2 and σ_y^2 are the variances, σ_{xy} is the covariance between x and y , and C_1, C_2 are small constants used to avoid numerical

instability. SSIM values are usually in the range from 0 to 1, where a value closer to 1 indicates higher structural similarity.

A complementary perceptual metric is Learned Perceptual Image Patch Similarity (LPIPS). LPIPS compares deep feature activations from a pretrained convolutional network for the original and reconstructed images, measuring perceptual similarity rather than pixel-wise error. It is often defined as a weighted distance between normalized feature maps from selected network layers:

$$[\text{NXB2}] \text{CongthucLPIPS} : \text{kiemkyhieu} w_l(\text{trong sokenh}) \text{vs} W_l(\text{chieu rong}) \text{---} \text{trungchu}' w/W' \text{denha} \quad (2.15)$$

where $\hat{f}_l(\cdot)$ is the unit-normalized feature map extracted from layer l of a pretrained network, (H_l, W_l) are the spatial dimensions of that layer, w_l are learned per-channel weights, and $\odot$ denotes element-wise multiplication. Lower LPIPS values indicate that the reconstructed image is closer to the original in terms of perceptually important structures and textures, making it especially relevant for GAN-based compression where visual quality is prioritized.

For the quantized HiFiC GAN accelerator, PSNR and SSIM provide useful objective indicators of reconstruction quality after quantization and hardware-oriented optimization. However, because GAN-based compression is designed to improve perceptual quality, these metrics should be interpreted together with perceptual evaluation and visual comparison. A reconstruction with slightly lower PSNR may still appear more natural if it preserves textures and perceptual details better than a distortion-optimized reconstruction. [?, ?, ?, ?]

2.6 Hardware Platform

The proposed accelerator is intended to be deployed on a Field-Programmable Gate Array (FPGA) platform. An FPGA is an integrated circuit that can be configured after manufacturing to implement a specific digital system. Unlike CPUs and GPUs, which execute instructions on a fixed architecture, an FPGA allows the designer to customize the hardware datapath, memory organization, and control logic according to the target algorithm. This flexibility makes FPGAs suitable for accelerating neural network workloads that require high parallelism, low latency, and efficient energy usage. [?, ?]

An FPGA device is mainly composed of configurable logic blocks, programmable interconnects, on-chip memory resources, and dedicated arithmetic units. Configurable logic blocks are used to implement control circuits and custom datapaths. Programmable interconnects connect these logic blocks and allow the designer to

build a complete hardware architecture. On-chip memory blocks, commonly referred to as Block RAM (BRAM), store intermediate feature maps, weights, partial sums, and temporary buffers. Dedicated Digital Signal Processing (DSP) slices are optimized for arithmetic operations such as multiplication and accumulation, which are dominant in convolutional neural networks and GAN generators.

For deep neural network acceleration, FPGAs provide several important advantages. First, they support fine-grained parallelism, allowing multiple processing elements to operate at the same time. This property is useful for convolutional and residual blocks, where many output pixels and channels can be computed independently. Second, FPGAs allow customized dataflow architectures that can increase data reuse and reduce external memory access. Since memory transfer often consumes significant energy and limits performance, keeping frequently used data in on-chip buffers is important for efficient accelerator design. Third, FPGAs can support different numerical precisions, such as fixed-point, integer, or floating-point formats, making them suitable for quantized neural network implementation.

In the context of the quantized HiFiC GAN accelerator, the FPGA platform is used to accelerate the computationally intensive generator network. The generator contains repeated convolutional and residual operations that can benefit from parallel multiply-accumulate units, pipelined execution, and local buffering. By mapping these operations to FPGA resources, the system can reduce inference latency and improve energy efficiency compared with executing the entire model on a general-purpose processor.

However, FPGA implementation also introduces several design constraints. The available number of logic elements, DSP slices, and BRAM blocks is limited, so the accelerator architecture must be carefully optimized to fit the target device. In addition, the memory bandwidth between external memory and the FPGA fabric can become a bottleneck if feature maps and weights are transferred frequently. Therefore, an efficient FPGA-based accelerator must balance parallelism, resource utilization, memory access, and operating frequency.

For this thesis, the hardware platform provides the physical foundation for hardware-software co-design. The software side prepares input data, controls execution, and manages communication with memory, while the FPGA accelerator performs the most time-consuming neural network computations. This division enables the system to preserve software flexibility while exploiting hardware-level parallelism for the quantized HiFiC GAN workload.

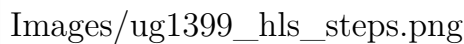
2.7 High-Level Synthesis

High-Level Synthesis (HLS) is an automated design process that converts a behavioral description written in C/C++ or SystemC into a register-transfer level (RTL) hardware implementation. According to the Vitis HLS user guide, the HLS flow starts from an algorithmic description, verifies it at C level, synthesizes RTL under clock and design constraints, verifies the generated RTL, and allows multiple architectures to be explored from the same source code [?].

2.7.1 Why HLS is Used

The main motivation for using HLS in this thesis is productivity. A ResBlock contains nested loops, convolution windows, channel accumulation, normalization, activation, and residual addition. Writing all these components directly in Verilog would require manual finite-state machines, address generators, pipeline registers, memory controllers, and interface logic. HLS allows the algorithm to be described closer to the C/C++ implementation, while pragmas guide the tool to generate the desired hardware micro-architecture.

Fig. 2.8 illustrates the main steps of HLS. The tool compiles the algorithm, schedules operations into clock cycles, binds operations to hardware resources, generates RTL, and then passes the design to logic synthesis. The important point is that HLS does not remove the need for hardware thinking. The designer still has to define the macro-architecture, memory hierarchy, parallelism, and interface protocol. Pragmas are the bridge between the algorithmic C/C++ description and the hardware architecture expected by the designer.



Images/ug1399_hls_steps.png

Figure 2.8 HLS compilation, scheduling, binding, and RTL generation flow [?]

Compared with traditional handwritten Verilog or VHDL, HLS provides a faster design-space exploration loop. The same C/C++ source can generate different RTL implementations by changing pragmas and constraints. This is valuable for CNN accelerators because the designer often has to trade latency against resource utilization. For example, increasing the unroll factor improves parallelism but consumes more DSP and LUT resources. Partitioning arrays improves memory bandwidth but consumes more BRAMs or registers.

Table 2.1 Comparison between HLS-based design and traditional handwritten RTL design

Aspect	HLS-based design	Handwritten Verilog/VHDL
Design entry	C/C++ algorithm with hardware-aware coding style and pragmas	Cycle-level RTL modules, FSMs, registers, and datapaths written manually
Development speed	Faster algorithm iteration and verification at C level	Slower but gives full control over every clock cycle and signal
Architecture exploration	Pragmas and constraints can generate multiple implementations from one source	Architecture changes often require significant RTL rewriting
Micro-architecture control	Controlled indirectly through scheduling, binding, interfaces, and memory pragmas	Controlled explicitly by the designer
Best suited for	Algorithm-dominated kernels such as convolution, filters, and dataflow pipelines	Highly specialized control paths, custom protocols, or manually optimized low-level blocks

2.7.2 Scheduling, Binding, and Generated Circuit Structure

HLS transforms an untimed C/C++ description into a timed RTL circuit through scheduling and binding. Scheduling determines which operations execute in each clock cycle based on data dependencies, clock period, resource availability, and user directives. Binding maps scheduled operations to concrete hardware resources such as DSP blocks, LUT-based adders, memories, multiplexers, and registers [?].

Fig. 2.9 shows a simplified scheduling and binding example. At the C level, the expression is written as ordinary arithmetic. During scheduling, the operations are assigned to clock cycles. During binding, multiplication and addition are mapped to functional units such as DSP or AddSub blocks. This is exactly where HLS pragmas affect the generated circuit: a pipeline pragma changes the timing schedule, an unroll pragma changes how many copies of the datapath are created, an array partition pragma changes the memory structure, and a bind operation pragma changes the resource selected for arithmetic.

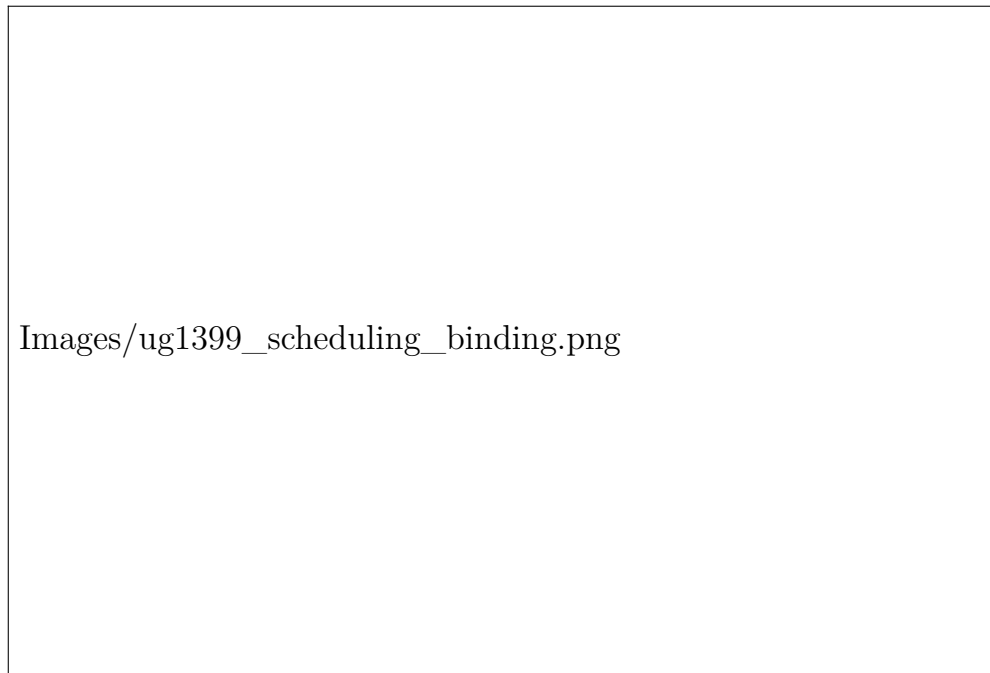


Figure 2.9 Scheduling and binding example [?]

For the HiFiC GAN ResBlock, this means the HLS code must be written with the final hardware in mind. The inner MAC loops should become pipelined datapaths. Channel or PE loops should become multiple parallel lanes. Buffers must provide enough memory ports to feed those lanes. Arithmetic operators should map to DSP blocks with predictable latency. The top-level function must expose AXI interfaces for PS–PL integration. Finally, the convolution pass and post-processing pass should overlap when possible.

2.7.3 Pipeline Pragma

The `#pragma HLS PIPELINE` directive reduces the initiation interval of a loop or function by allowing operations from different loop iterations to execute concurrently. Without pipelining, a loop iteration must complete before the next iteration starts. With pipelining, the loop is divided into stages separated by registers. After the pipeline is filled, a new iteration can begin every `II` cycles.

When $II=1$ is achieved, the loop accepts new input every clock cycle.

Fig. 2.10 demonstrates this timing change. Without loop pipelining, read, compute, and write operations are serialized across iterations. With loop pipelining, these operations overlap, reducing the interval between consecutive reads from three cycles to one cycle in the illustrated example [?].

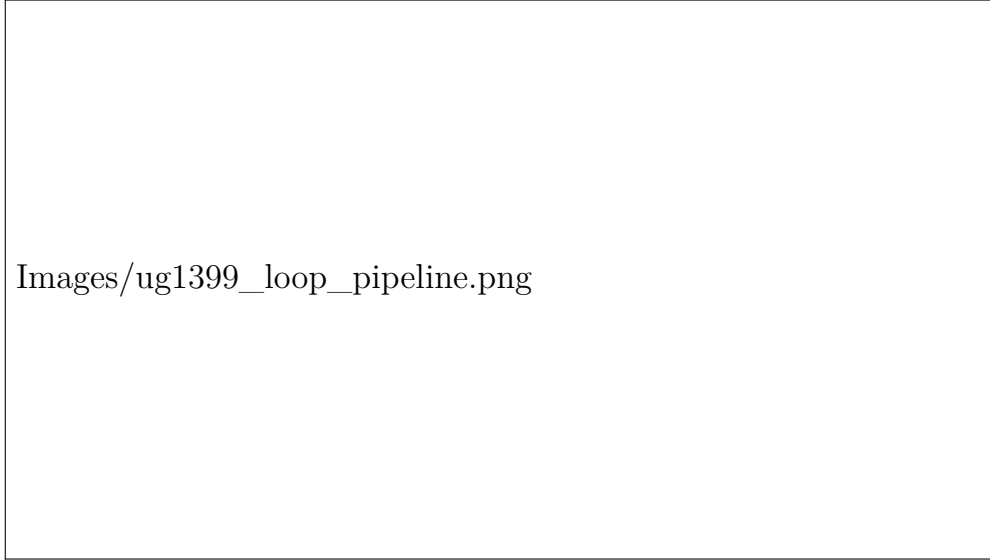


Figure 2.10 Loop pipeline timing example [?]

In the proposed ResBlock, PIPELINE $II=1$ is used for inner MAC loops inside each PE. At the circuit level, this creates pipeline registers along the multiplication and accumulation path and schedules the datapath so that different convolution windows are processed in overlapped clock cycles. The timing benefit is higher throughput: after the initial latency, the PE can produce one partial or final result per cycle if memory ports and dependencies do not stall the loop. This pragma is therefore most suitable for loops with regular data access and high iteration counts, such as convolution accumulation.

2.7.4 Unroll Pragma

The `#pragma HLS UNROLL` directive transforms a loop by creating multiple copies of the loop body in hardware. If the loop is fully unrolled, all iterations are implemented as parallel hardware. If a factor N is specified, N copies of the loop body are created and the number of sequential loop iterations is reduced accordingly [?].

The hardware effect of partial loop unrolling is illustrated in Listings 1 and 2. A rolled loop reuses one datapath across multiple cycles, while an unrolled loop replicates the loop body so that several iterations can execute in parallel. The

unroll factor does not have to be an integer divisor of the maximum loop iteration count. Therefore, when the trip count is not known exactly, Vitis HLS adds an exit check to preserve the same functionality as the original loop.

Listing 1: Pseudo-code before partial loop unrolling

```
for (int i = 0; i < X; i++) {
    #pragma HLS UNROLL factor=2
    a[i] = b[i] + c[i];
}
```

Listing 2: Equivalent pseudo-code after unrolling by factor 2

```
for (int i = 0; i < X; i += 2) {
    a[i] = b[i] + c[i];
    if (i + 1 >= X) break;
    a[i + 1] = b[i + 1] + c[i + 1];
}
```

At the circuit level, unrolling changes a single reused datapath into multiple datapaths operating in parallel. For the HiFiC ResBlock, this is used in PE loops and SIMD channel lanes. Instead of one multiplier–adder chain serving all lanes sequentially, the unrolled design instantiates multiple multiplier–adder chains. The timing benefit is reduced latency and increased spatial parallelism. However, resource usage grows because more DSP blocks, LUTs, registers, and routing are required. If the designer knows that the unroll factor is an integer divisor of the maximum iteration count, the `skip_exit_check` option can remove the exit check and its associated control logic. UNROLL is therefore suitable when the target FPGA has enough resources and when the memory system can supply all parallel lanes.

2.7.5 Array Partition Pragma

The `#pragma HLS ARRAY_PARTITION` directive partitions an array into smaller arrays or individual elements. According to UG1399, this increases the number of read and write ports available to the generated RTL and can improve throughput, at the cost of using more memory instances or registers [?].

This pragma is essential when UNROLL is used. A conventional BRAM has a limited number of ports, so multiple PEs cannot read all required operands in the same cycle if the data remain in a single memory. By partitioning the input feature buffer, weight buffer, or output feature map array, HLS maps the storage into multiple banks or registers. At the circuit level, this removes memory port conflicts. At the timing level, it allows the pipelined and unrolled datapaths to sustain II=1 rather than stalling while waiting for memory access. ARRAY_PARTITION is

therefore suitable for small to medium on-chip buffers that feed parallel PEs.

Listing 3 shows a basic block-partition example from the HLS style of coding. The original array AB[13] is split into four smaller arrays using contiguous blocks of elements. This does not change the logical indexing in the C/C++ program, but it changes the generated hardware storage: instead of one memory with limited ports, the tool can create several memory banks that may be accessed in parallel when the access pattern targets different banks.

Listing 3: Pseudo-code for block partitioning a 13-element array

```
int AB[13];
#pragma HLS ARRAY_PARTITION variable=AB type=block factor=4
```

In the proposed generator accelerator, the same idea is applied to buffers that feed parallel PE lanes, as shown in Listing 4. The lane dimension is partitioned so each unrolled PE can read its own activation and weight values without waiting for a shared BRAM port. This is why ARRAY_PARTITION is often paired with UNROLL: unrolling creates parallel datapaths, while partitioning provides the memory bandwidth needed to keep those datapaths busy.

Listing 4: Pseudo-code for array partitioning to feed parallel PE lanes

```
# Partition the lane dimension into independent banks
#pragma HLS ARRAY_PARTITION variable=ifm_buffer complete dim=1
#pragma HLS ARRAY_PARTITION variable=weight_buffer complete dim=1

for pe = 0 to PE_UNROLL - 1 do
    #pragma HLS UNROLL
    pixel = ifm_buffer[pe][row][col]
    weight = weight_buffer[pe][kernel]
    partial[pe] = pixel * weight
end for
```

2.7.6 Bind Operation Pragma

The #pragma HLS BIND_OP directive controls how a specific arithmetic operation is implemented in hardware. UG1399 states that operations such as multiplication and addition can be bound to implementation resources such as fabric or DSP, and the operation latency can also be specified when supported [?].

Listing 5 shows a basic example of this directive. The multiplication assigned to variable c is explicitly implemented as a two-stage pipelined multiplier in FPGA fabric. The second multiplication that produces d is not bound, so Vitis HLS remains free to choose its implementation automatically. This example demonstrates that BIND_OP is applied to a selected result variable, rather than

globally changing every multiplication in the function.

Listing 5: Pseudo-code for binding one multiplication to fabric logic

```
int foo(int a, int b) {  
    int c, d;  
    #pragma HLS BIND_OP variable=c op=mul impl=fabric latency=2  
    c = a * b;  
    d = a * c;  
    return d;  
}
```

In the generator accelerator, multiplication dominates convolution. Mapping FP16 multiplication to DSP resources makes the implementation faster and more predictable than relying entirely on LUT fabric. At the circuit level, `BIND_OP` fixes the resource used by selected arithmetic operators and can introduce a defined multi-cycle latency. At the timing level, this helps timing closure because the critical path through floating-point multiplication can be broken into known pipeline stages. For half-precision data, the operation can be treated as an `hmul` operation and mapped to a DSP-based implementation such as `fulldsp`, as illustrated in Listing 6. This pragma is suitable for compute-heavy arithmetic kernels where the designer wants explicit control over DSP utilization and operator latency.

Listing 6: Pseudo-code for binding FP16 multiplication to DSP resources

```
for k = 0 to KERNEL_SIZE - 1 do  
    #pragma HLS PIPELINE II=1  
    #pragma HLS BIND_OP variable=product op=hmul impl=fulldsp latency=3  
    product = input_value[k] * weight_value[k]  
    acc = acc + product  
end for
```

2.7.7 Interface Pragma

The `#pragma HLS INTERFACE` directive specifies how RTL ports are created from top-level function arguments. The pragma must be placed inside the boundary of the top-level function and is applied to the argument named by `port=<name>` or to the function return port. In general form, it selects an interface mode and then optionally configures details such as bundle grouping, address offset, or direct I/O behavior [?].

Listing 7: General syntax of the HLS interface pragma

```
#pragma HLS INTERFACE mode=<mode> port=<name> direct_io=<true|false> [  
    OPTIONS]
```

The mode option can be understood in three groups:

- **Port-level protocols:** These modes describe simple RTL ports for scalar, array, or FIFO-style arguments.
 - `ap_none`: Creates a plain data port without an additional handshake signal.
 - `ap_vld`: Adds a valid signal to indicate that the data value is valid.
 - `ap_ack`: Adds an acknowledge signal to indicate that the data has been accepted.
 - `ap_hs`: Combines valid and acknowledge signals to create a two-way handshake.
 - `ap_ovld`: Adds a valid signal for output data ports.
 - `ap_memory` and `bram`: Implement array arguments as RAM-style interfaces.
 - `ap_fifo`: Implements a read-only or write-only argument as a FIFO-style interface.
 - `direct_io`: Forces a function argument to be implemented as direct I/O when required.
- **AXI interface protocols:** These modes are used when the HLS kernel must connect to a system bus or data stream.
 - `s_axilite`: Maps arguments to AXI4-Lite registers for configuration, scalar parameters, base addresses, and control/status access from the processor.
 - `m_axi`: Creates an AXI4 master interface so the accelerator can issue burst reads and writes to external memory.
 - `axis`: Creates an AXI4-Stream interface for streaming data movement without address channels.
- **Block-level control protocols:** These modes define how the whole accelerator starts, finishes, and reports its state.
 - `ap_ctrl_hs`: Provides start, done, idle, and ready control signals.
 - `ap_ctrl_chain`: Adds chaining support for back-to-back kernel execution.
 - `ap_ctrl_none`: Removes block-level control for a free-running kernel.

For the proposed accelerator, the top-level `ResBlock` function must communicate with the Zynq PS and DDR memory. `m_axi` interfaces are suitable for high-throughput feature-map and weight transfers, while `s_axilite` is suitable for

control registers such as base addresses, tensor dimensions, and start/done signals. At the circuit level, this pragma generates AXI adapters, address channels, data channels, and control ports. At the timing level, it affects system throughput by enabling burst transfers and reducing software–hardware communication overhead. INTERFACE is therefore suitable for top-level kernel integration rather than internal arithmetic optimization.

Listing 8: Pseudo-code for ResBlock top-level AXI interfaces

```
void resblock_top(fp16 *ifm, fp16 *weight, fp16 *ofm, int height, int width) {
    #pragma HLS INTERFACE mode=m_axi port=ifm      offset=slave bundle=gmem0
    #pragma HLS INTERFACE mode=m_axi port=weight  offset=slave bundle=gmem1
    #pragma HLS INTERFACE mode=m_axi port=ofm      offset=slave bundle=gmem2
    #pragma HLS INTERFACE mode=s_axilite port=ifm    bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=weight bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=ofm    bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=height bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=width  bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=return bundle=control

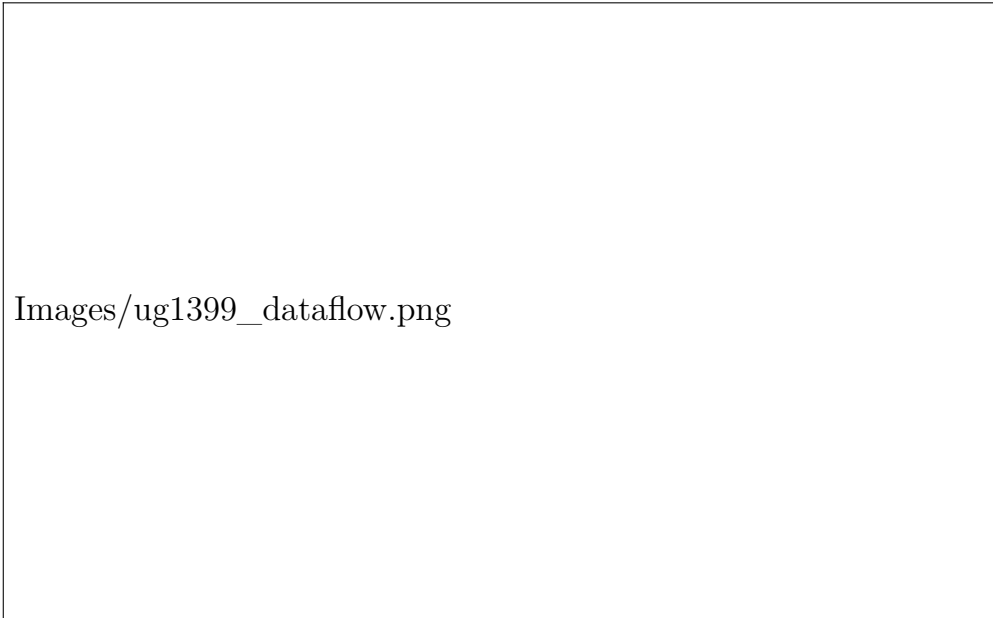
    run_resblock_pipeline(ifm, weight, ofm, height, width)
}
```

Listing 8 uses three `m_axi` ports for `ifm`, `weight`, and `ofm`. These pointer arguments represent large tensors stored in DDR memory. Assigning them to separate bundles, `gmem0`, `gmem1`, and `gmem2`, allows HLS to generate separate AXI master adapters, so input feature-map reads, weight reads, and output feature-map writes can be scheduled with less port contention. The `offset=slave` option indicates that the base address of each pointer is programmed through the control interface.

The same pointer arguments also appear as `s_axilite` ports in the control bundle so that software running on the PS can write their base addresses before starting the accelerator. The scalar parameters `height` and `width` are also mapped to `s_axilite` registers because they are configuration values rather than high-bandwidth data streams. Finally, `port=return` creates the AXI4-Lite control/status interface for starting the kernel and observing completion. In system terms, software writes the base addresses and dimensions through AXI4-Lite, starts the kernel, and the PL kernel then uses AXI master ports to move bulk feature-map and weight data to and from DDR memory.

2.7.8 Dataflow Pragma

The `#pragma HLS DATAFLOW` directive enables task-level pipelining. It allows functions or loops to overlap in time by inserting channels such as FIFOs or ping-pong memories between producer and consumer tasks. As shown in Fig. 2.11, dataflow pipelining can reduce total latency because one function can start before the previous function has completely finished all of its work [?].



Images/ug1399_dataflow.png

Figure 2.11 Task-level dataflow timing example [?]

In the proposed design, DATAFLOW is suitable for overlapping coarse-grained ResBlock stages. For example, Pass A performs convolution and writes intermediate output feature maps, while Pass B performs normalization, activation, and residual addition. When these stages are connected through FIFO or ping-pong buffers, the second stage can consume data while the first stage continues producing later rows or tiles. At the circuit level, DATAFLOW creates concurrently active processes connected by channels. At the timing level, it reduces idle cycles between stages and improves end-to-end throughput, provided that the producer and consumer rates are balanced.

2.7.9 Summary of HLS Pragma Selection

Table 2.2 summarizes the role of each pragma in the proposed generator accelerator. The key observation is that pragmas must be selected together. A pipelined MAC loop cannot reach $II=1$ if the memory has insufficient ports. An unrolled PE cluster cannot be useful if the interface cannot supply data. Dataflow overlap is effective only when buffers decouple producer and consumer stages.

Table 2.2 Summary of HLS pragmas used in the proposed generator accelerator

Pragma	Circuit effect	Timing effect	Suitable use in this thesis
PIPELINE II=1	Adds pipeline scheduling and registers to loop/function datapaths	Starts a new iteration every cycle when dependencies allow	Inner MAC loops in each PE
UNROLL factor=N	Creates multiple copies of loop hardware	Reduces latency and increases parallel work per cycle	PE loops and SIMD channel lanes
ARRAY_PARTITION	Splits arrays into memory banks or registers	Removes memory port conflicts and supports II=1	IFM buffers, weight buffers, and OFM arrays
BIND_OP	Maps arithmetic to selected resources such as DSP or fabric	Controls operator latency and helps timing closure	FP16 multiplication inside PEs
INTERFACE	Generates AXI/control ports for top-level arguments	Enables efficient PS-PL and DDR communication	ResBlock top-level kernel connection
DATAFLOW	Creates concurrent tasks connected by FIFO/PIPO channels	Overlaps stages and reduces idle cycles	Convolution pass and normalization/activation pass

Overall, HLS is appropriate for this thesis because the target workload is algorithmically complex but structurally regular. The designer can express the HiFiC ResBlock in C/C++, verify functionality at a high level, and use pragmas to control parallelism, memory bandwidth, arithmetic binding, interfaces, and task overlap. This approach provides a practical compromise between software-level productivity and hardware-level performance.

2.8 Software Tools for Verification and Synthesis

When designing FPGA-based accelerators for CNN workloads, verification and synthesis tools are essential for ensuring functional correctness, resource efficiency, and timing feasibility. The tool flow must support both algorithm-level validation and hardware-level implementation. In this thesis, the main software tools are Vivado for FPGA synthesis and system integration, Vitis for embedded software development, and PetaLinux for preparing the Linux-based runtime environment on the Zynq Processing System.

2.8.1 Vivado

Vivado is the FPGA design suite developed by Xilinx, now part of AMD, for synthesis, implementation, timing analysis, IP integration, and bitstream generation targeting Xilinx FPGA devices. After the HLS kernel has been verified and exported as an RTL IP core, Vivado is used to integrate the accelerator into the complete FPGA system.

In the proposed design flow, Vivado maps the generated RTL into FPGA resources such as LUTs, flip-flops, BRAMs, URAMs, and DSP slices. The synthesis stage converts the RTL description into a gate-level netlist, while the implementation stage performs placement and routing on the target device. Timing analysis is then used to check whether the design satisfies the required clock constraints. These steps are necessary for evaluating whether the generator accelerator can meet the target frequency and resource budget on the selected FPGA platform.

Vivado also provides IP Integrator and reusable IP cores for building a complete Zynq-based system. The accelerator IP can be connected to the Processing System, DDR memory controller, AXI interconnects, reset logic, and clocking blocks through a block design. After verification and implementation, Vivado generates the bitstream used to configure the Programmable Logic. In this thesis, Vivado therefore acts as the bridge between the HLS-generated accelerator and the deployable FPGA system.

2.8.2 Vitis

Vitis is AMD’s unified software development platform for building, debugging, and deploying applications on heterogeneous Xilinx devices. In a Zynq-based system, Vitis is used after the hardware platform has been created in Vivado. The exported hardware description provides information about the Processing System, Programmable Logic, memory map, AXI interfaces, interrupts, and available hardware IP cores. Based on this platform, Vitis can generate the board support package and compile software that runs on the ARM processors in the PS.

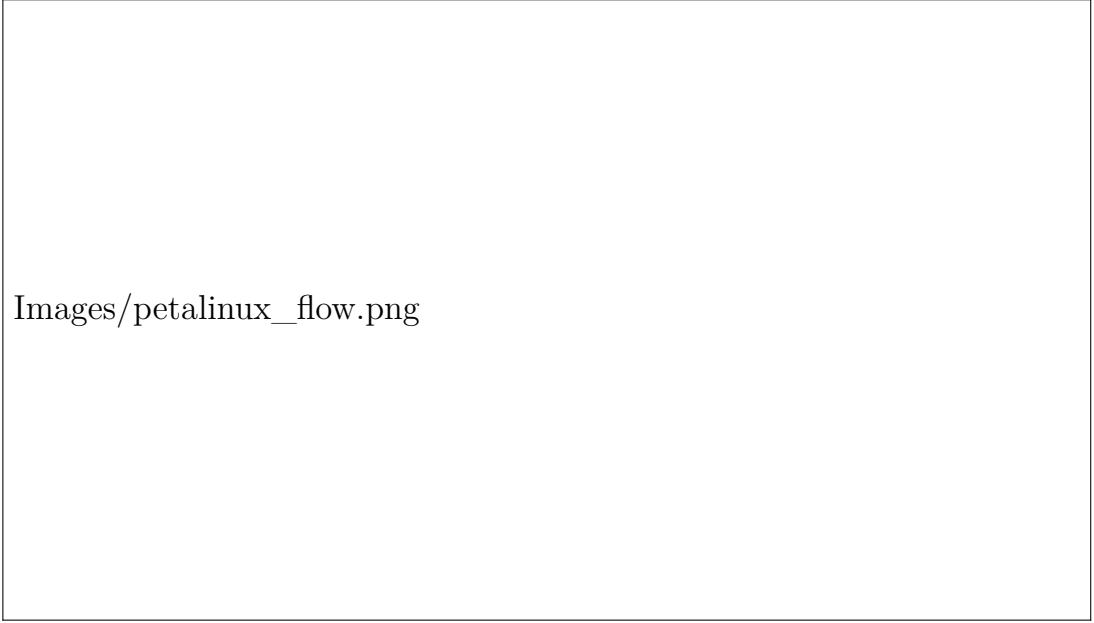
For the proposed HiFiC GAN accelerator, Vitis is used to develop the host application that controls the hardware kernel. The software configures accelerator registers through AXI4-Lite, writes base addresses and tensor parameters, starts the accelerator, waits for completion, and reads back status or performance counters. It can also be used to measure end-to-end execution time and compare hardware acceleration against software execution. Therefore, Vitis connects the generated hardware accelerator with the software layer that manages data movement and execution control.

Vitis is also useful for debugging embedded applications because it supports compilation, launching, and step-by-step inspection on the target board or emulator. In this design flow, Vitis complements Vitis HLS: Vitis HLS focuses on generating the accelerator IP from C/C++ code, while Vitis focuses on writing and validating the PS-side application that uses that IP in the complete embedded system.

2.8.3 PetaLinux

PetaLinux is an AMD/Xilinx embedded Linux development environment for creating, configuring, building, and booting Linux images for FPGA SoC platforms. It provides command-line tools and build recipes that package the hardware description, bootloader, device tree, Linux kernel, root file system, and user applications into a deployable embedded Linux system. For Zynq UltraScale+ platforms, PetaLinux is especially useful because the PS can run Linux while the PL implements custom accelerator IP.

Fig. 2.12 shows a typical PetaLinux development flow. The process starts from the hardware platform exported by Vivado. The designer then creates or selects a PetaLinux project using `petalinux-create`. Next, `petalinux-config` is used to import the hardware description and configure kernel options, root file-system packages, user applications, device-tree settings, and system parameters. After configuration, `petalinux-build` compiles the boot components, Linux kernel, device tree, and root file system. The optional `petalinux-package` step can package boot artifacts such as the boot image. Finally, `petalinux-boot` or an SD-card boot flow is used to run the generated Linux image on QEMU or on the target board.



Images/petalinux_flow.png

Figure 2.12 PetaLinux project creation, configuration, build, package, and boot flow

In the proposed system, PetaLinux provides the runtime environment for controlling the accelerator from software. The generated Linux image can include the required drivers, device-tree entries, libraries, and application files needed to access the accelerator IP and DDR memory from user space or kernel space. This makes PetaLinux the final deployment layer after the accelerator has been generated by Vitis HLS, integrated in Vivado, and controlled by software developed in Vitis.

2.9 Chapter Conclusion

Chapter 2 presented the theoretical background required for designing the proposed quantized HiFiC GAN accelerator. It first reviewed the main CNN operations used in learned image compression, including convolution, activation, normalization, residual connections, and up-sampling, and then introduced GANs and the HiFiC architecture together with the mixed-precision quantization concept used in this thesis. It also presented the evaluation metrics for both hardware efficiency and image-compression quality. These concepts explain the computational and memory-access characteristics that must be considered when mapping the ResBlock workload onto FPGA hardware.

The chapter then introduced the FPGA platform and explained why FPGA resources such as LUTs, flip-flops, on-chip memories, DSP slices, and AXI interfaces are suitable for accelerating CNN-based image compression. It also discussed the HLS design methodology, including scheduling, binding, pipelining, loop unrolling, array partitioning, operation binding, interface generation, and task-level dataflow.

Finally, the chapter summarized the software tools used in the development flow, including Vitis HLS for accelerator generation, Vivado for FPGA synthesis and system integration, Vitis for embedded software development, and PetaLinux for building the Linux-based deployment environment.

Based on this background, the following chapters describe how the proposed generator accelerator is designed, optimized, integrated, and evaluated on the target FPGA platform.

CHAPTER 3. IMPLEMENTATION

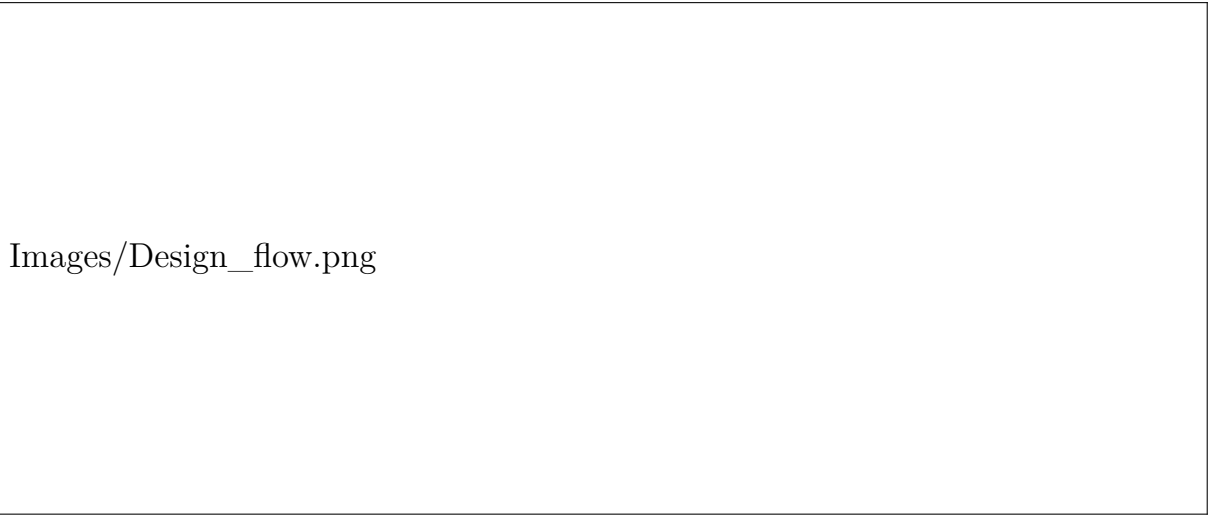
Chapter 3 presents the implementation of the proposed quantized HiFiC GAN accelerator. The discussion begins with the system-level architecture and hardware–software partitioning, then presents the mixed-precision quantization strategy applied to the HiFiC model and the workload analysis that identifies the residual block as the main acceleration target. It then details the internal accelerator blocks and finally describes how the design is prepared for deployment on the ZCU104 FPGA platform.

The goal of this chapter is not to redefine the HiFiC algorithm, which has already been introduced in the theoretical background, but to explain how the model is mapped into a practical hardware–software co-design. Therefore, the chapter emphasizes architectural partitioning, mixed-precision quantization, data movement, buffering, parallel computation, and the High-Level Synthesis (HLS) flow used to generate the FPGA accelerator.

3.1 System-Level Architecture Design

This section describes how the design specifications introduced in Section 1.3.1 of Chapter 1 – a quantized HiFiC GAN generator accelerator on the ZCU104 platform, modeled in C/C++ and built with the Vitis HLS and Vivado tool flow – are realized as a concrete hardware–software system.

Figure 3.1 shows the overall design flow followed in this thesis, organized into three phases: *specification and analysis*, *design and verification*, and *synthesis and integration*.



Images/Design_flow.png

Figure 3.1 Overall design flow of the HiFiC GAN accelerator

In the **specification and analysis** phase, the problem is defined together with the HiFiC GAN specification, followed by a state-of-the-art analysis and a system specification analysis. These steps lead to a hardware–software co-design architecture proposal that decides which parts of the inference pipeline are accelerated on the FPGA and which remain in software.

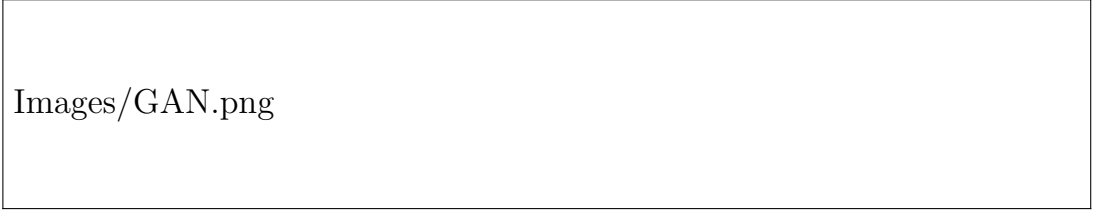
In the **design and verification** phase, the selected compute kernels are written as a C/C++ algorithm in Vitis HLS. In parallel, the pre-trained HiFiC model provides golden data for a C/C++ testbench environment. The HLS design is checked by C simulation against these golden outputs; if the results do not match, the C/C++ design is revised and the loop is repeated until functional correctness is achieved.

In the **synthesis and integration** phase, the verified design is passed through High-Level Synthesis, which produces a synthesis report and enables C/RTL co-simulation to confirm that the generated RTL behaves identically to the C model. If the design does not meet the target specifications, the C/C++ description is tuned – through loop pipelining, array partitioning, on-chip buffering, and interface directives – and re-synthesized. Once the targets are met, the IP core is exported and integrated into a Zynq UltraScale+ system using Vivado, where it is connected to external DDR memory and the Processing System (PS) through AXI interfaces and controlled from a Vitis software application.

3.1.1 HiFiC GAN Model-Level Data Path

This subsection first clarifies the model-level data path before discussing hardware partitioning. The purpose is to identify which part of the HiFiC inference pipeline should remain in software and which part should be mapped to the FPGA accelerator.

The overall HiFiC architecture used in this thesis was presented in Figure 2.7 of Chapter 2. The model consists of an encoder network, a hyperprior network, and a generator network. The encoder transforms the input image into a compact latent representation, while the hyperprior network provides side information for entropy modeling. The generator reconstructs the final image from the latent representation and is responsible for recovering high-quality visual details.



Images/GAN.png

Figure 3.2 Generator structure with residual blocks and up-convolution blocks

Therefore, the system-level design decision is derived directly from the model data path: software keeps the compression front-end (encoder, hyperprior, and entropy coding), while the entire generator (decoder) is offloaded to hardware. This relationship is the basis for the hardware–software partitioning described next.

3.1.2 System Partitioning and PS–PL Interfaces

Based on the model-level analysis in the previous subsection, the proposed implementation follows a hardware–software co-design strategy. Tasks that require flexibility, irregular control, file handling, or model orchestration – namely the encoder, hyperprior, and entropy coding of the compression front-end – are executed on the Processing System (PS). The entire generator (decoder), which dominates the reconstruction computation, is offloaded to the Programmable Logic (PL) as a single accelerator IP, where parallel datapaths and streaming buffers are customized for the convolution-heavy workload.

Figure 3.3 summarizes this partitioning. The PS manages model control, parameter preparation, memory allocation, and accelerator invocation. The PL executes the generator accelerator as a single reusable compute IP. Large tensors are exchanged through shared DDR memory, while lightweight control signals are transferred through memory-mapped AXI registers.

Images/HW_SW.jpg

Figure 3.3 System-level hardware–software co-design overview of the HiFiC inference pipeline

The PS–PL communication is organized through a small number of well-defined interfaces, as summarized in Table 3.1. This interface view makes the system partition explicit and connects the high-level architecture in Figure 3.3 with the detailed generator accelerator design presented later in this chapter.

Table 3.1 System-level interfaces used by the generator accelerator

Interface	Direction	Role in the system
AXI4-Lite control	PS $\rightarrow$ PL	Transfers configuration values such as tensor dimensions, base addresses, start command, and status polling.
AXI master read	DDR $\rightarrow$ PL	Loads input feature maps, convolution weights, bias values, and normalization parameters from shared memory.
AXI master write	PL $\rightarrow$ DDR	Writes the reconstructed output feature maps of the generator back to shared memory.
Shared DDR buffers	PS $\leftrightarrow$ PL	Provide the common storage space used to exchange large tensors between software execution and hardware acceleration.

From the software perspective, the accelerator call behaves as a coarse-grained operation. The PS prepares the input and parameter buffers in DDR, writes the required configuration values through AXI4-Lite, starts the accelerator, waits for completion, and then reads back the reconstructed image. From the hardware perspective, the PL implements the complete generator datapath with memory-mapped input and output ports, reusing the same MAC engines across the residual blocks and the other generator stages.

3.2 Mixed-Precision Quantization of HiFiC Model

The deployment of learned image compression models such as HiFiC on FPGA platforms introduces significant challenges related to memory bandwidth, storage capacity, and computational efficiency. To address these constraints, a mixed-precision quantization strategy is adopted, aiming to reduce numerical precision while preserving compression performance and perceptual quality.

Figure 3.4 illustrates the proposed quantization architecture applied to the HiFiC model. The quantization process is designed to operate in a post-training manner, eliminating the need for retraining and enabling rapid deployment on the target hardware.



Figure 3.4 Mixed-precision quantization architecture applied to convolutional layers

3.2.1 Quantization Strategy

A post-training mixed-precision quantization approach is employed, where the internal computation of the neural network is performed using 16-bit floating-point (FP16) precision, while maintaining 32-bit floating-point (FP32) precision at the input and output interfaces. This strategy balances computational efficiency with numerical stability and compatibility with existing preprocessing and postprocessing pipelines.

The quantization strategy consists of the following key principles:

- **Interface Preservation:** The input and output tensors of the HiFiC model remain in FP32 format to ensure compatibility with external modules.
- **Internal Precision Reduction:** All intermediate feature maps and arithmetic operations within the network are executed in FP16 precision.
- **Static Weight Casting:** Convolutional kernels, bias terms, and learned parameters are statically converted from FP32 to FP16 to reduce memory footprint.

To enforce this precision boundary, explicit casting operations are inserted immediately after the input layer (FP32 to FP16) and before the output layer (FP16 to FP32). Redundant casting operators introduced during automatic model conversion are identified and removed to maintain a continuous FP16 dataflow throughout the network.

3.2.2 Operator-Level Precision Mapping

Following quantization, the HiFiC model executes the majority of its operations within the FP16 domain. These operations can be categorized into two main groups:

- **Computational Operations:** Convolution (Conv), transposed convolution (ConvTranspose), and matrix multiplication layers dominate the arithmetic workload of the Generator and are executed entirely in FP16 precision. This enables efficient utilization of FPGA DSP resources for multiply-accumulate (MAC) operations.
- **Non-linear and Element-wise Operations:** Activation functions (ReLU), element-wise addition, subtraction, multiplication, division, and exponential functions are also performed in FP16 precision, ensuring consistency across the computation graph.

By enforcing FP16 computation across all internal operators, the model achieves a significant reduction in memory bandwidth consumption and computational complexity, while preserving the structural integrity of the HiFiC architecture.

3.2.3 Quantization Rationale

The choice of FP16 precision is motivated by its favorable trade-off between numerical range and hardware efficiency. Compared to integer-based quantization schemes, FP16 offers higher dynamic range and improved stability for GAN-based architectures, where small numerical perturbations can otherwise lead to visible degradation in reconstructed images.

This mixed-precision quantization framework establishes a robust foundation for hardware acceleration of the HiFiC model on FPGA platforms, enabling efficient execution without compromising compression fidelity or perceptual quality.

3.3 Workload Analysis and Acceleration Target

To determine the most suitable hardware acceleration target, the generator network was profiled on a CPU platform. The execution time of each generator block was measured on an Intel Core i9 processor operating at 2.2 GHz. The results are summarized in Table 3.2.

Table 3.2 Runtime breakdown of HiFiC generator blocks executed on Intel Core i9 CPU @ 2.2 GHz

Block Name	Execution Time (s)	Percentage (%)
conv_block_init	23.139	2.16
res_block_0	111.327	10.37
res_block_1	102.859	9.58
res_block_2	97.376	9.07
res_block_3	96.871	9.03
res_block_4	96.799	9.02
res_block_5	102.545	9.55
res_block_6	106.396	9.91
res_block_7	113.640	10.59
res_block_8	118.866	11.08
upconv_blocks (Total 1–4)	87.696	8.17
conv_block_out	15.588	1.45
Others (Add, Overhead)	0.088	0.03
Total Runtime	1073.19	100

The ResBlocks are also attractive from a hardware design perspective because they share a regular computation pattern. Each block is mainly composed of convolution, normalization, activation, and residual addition. The convolution layers reuse the same local-window structure across many spatial positions and channels, which matches the strengths of FPGA architectures: local buffering, parallel MAC units, and pipelined execution.

Although the residual blocks dominate the runtime, the proposed accelerator implements the *complete* generator on hardware, so that the entire reconstruction path runs on the FPGA without returning intermediate feature maps to software. The compression front-end (encoder, hyperprior, and entropy coding) remains in software. This scope keeps the heaviest computation – the residual blocks – on dedicated hardware while still producing the final image entirely within the Programmable Logic.

3.4 Detailed Internal Block Design

This section describes the internal architecture of the proposed accelerator. The entire HiFiC generator is implemented as a single HLS IP, `full_generator_top`, organized into three main cores: the **Fusion Core** (executes the initial convolution block, nine residual blocks, and the global skip addition), the **UpConv Block Core** (executes four transposed-convolution up-sampling blocks), and the **Conv77 Core** (executes the final 7×7 convolution that produces the RGB image). Packing the whole generator into one IP allows the multiply-accumulate (MAC) engines to be reused across cores and simplifies system integration, at the cost of a tight on-chip resource budget. The explanation below begins with the overall accelerator architecture, then describes the common building blocks (data buffering and processing elements), and finally details the implementation of each core.

3.4.1 Overall Generator Accelerator Architecture

At the top level, the generator accelerator is organized into three cores, as shown in Figure 3.5: the **Fusion Core**, the **UpConv Block Core**, and the **Conv77 Core**. Each core is an independent hardware module that communicates with external DDR memory through its own pair of AXI4 master interfaces – one for reading input feature maps, weights, and parameters, and one for writing the resulting output feature maps. The cores are invoked sequentially under software control to realize the complete generator datapath. Internally, every core reuses the same convolution-then-normalization-and-activation motif, which is why a small number of hardware engines can implement the whole generator network. Because the Fusion Core is the largest and most representative of this pattern, its internal

datapath is described in detail first, and the lighter UpConv Block and Conv77 cores are then presented as variations of it.

Images/GAN_accelarator.png

Figure 3.5 Overall architecture of the HiFiC GAN accelerator

3.4.2 Fusion Core

The Fusion Core executes the initial convolution block (CBI), the nine residual blocks, and the global skip addition. Its computational pattern is the residual block shown in Figure 3.6: two 3×3 convolutions, each followed by normalization, with a ReLU between them and a skip connection that adds the block input to its output. The remainder of this subsection presents the core bottom-up: first the data-feeding building blocks (line buffer, window, and boundary handling), then the processing element that performs the arithmetic, then how these are assembled into a two-pass convolution-and-normalization datapath, and finally the complete Fusion Core block diagram.

Images/resblock.png

Figure 3.6 Computational flow of a residual block in the HiFiC generator

Line Buffer, Window, and Boundary Handling

The convolution engine processes convolution windows in a streaming manner. Instead of repeatedly loading the same pixels or feature values from external memory, the design stores recently used rows in on-chip line buffers. For a 3×3

convolution, two previous rows are sufficient because the current row is provided by the input stream.



Figure 3.7 Two-line buffer structure for streaming convolution

Figure 3.7 illustrates the two-line buffer organization. As new feature values arrive in raster-scan order, the buffer keeps the spatial relationship between adjacent rows. This allows the accelerator to construct valid convolution windows without reloading the same rows from DDR memory.

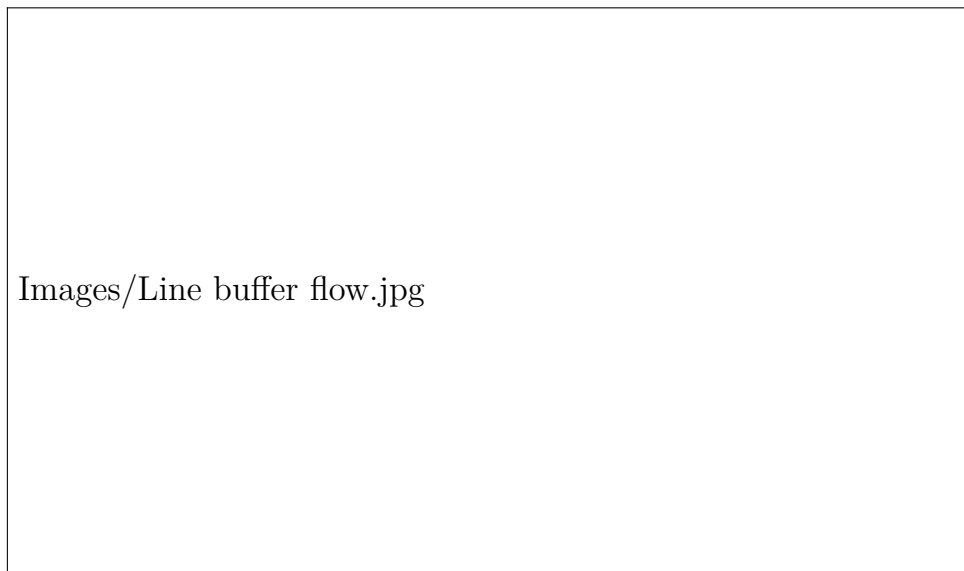


Figure 3.8 Two-line buffer dataflow and update mechanism

The update behavior is shown in Figure 3.8. At each new input position, stored values are shifted to keep the previous rows aligned with the current row. This structure converts external memory traffic into local buffer reuse, which is essential for high-throughput convolution on FPGA.

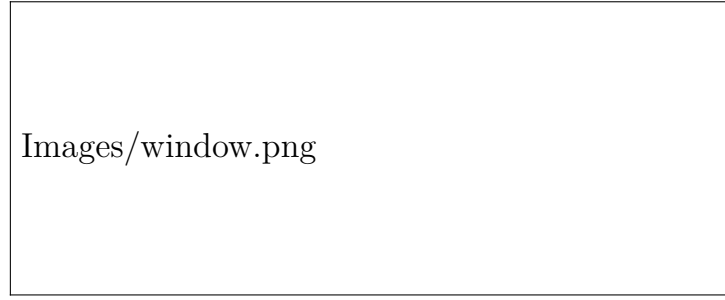


Figure 3.9 Sliding window structure for convolution processing

After the line buffer prepares neighboring rows, a sliding window extracts the local 3×3 region required by each convolution operation. The window structure in Figure 3.9 provides the input values for the PE cluster. In the proposed design, multiple window positions on the same row can be generated in parallel so that several output pixels are computed at the same time.

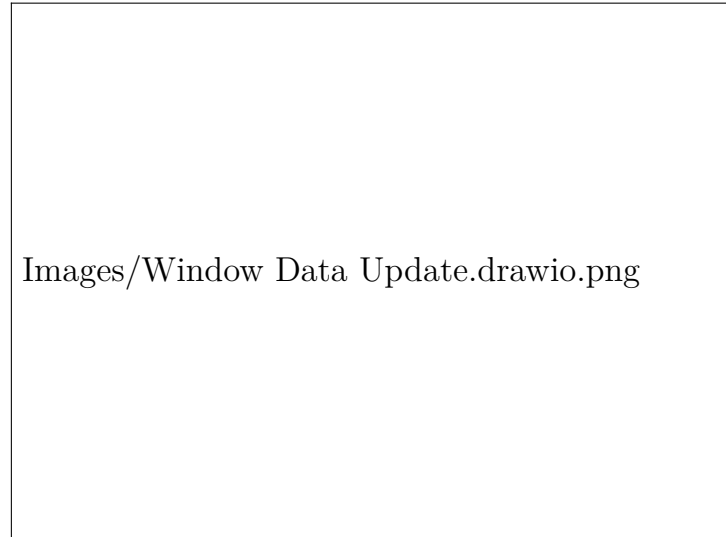


Figure 3.10 Parallel window update mechanism with vertical-only movement

Figure 3.10 shows the window update mechanism used to support parallel processing. Rather than moving a single window across the row one position at a time, the architecture assigns multiple horizontal positions to different processing lanes. The input stream then advances to the next row, while each lane maintains the correct local neighborhood for its assigned output position.

Boundary handling is implemented using reflection padding, as shown in Figure 3.11. Reflection padding mirrors feature values at the border, preserving local continuity near image boundaries. This is suitable for image reconstruction workloads because it avoids the abrupt border discontinuities that may be introduced

by simple zero padding.

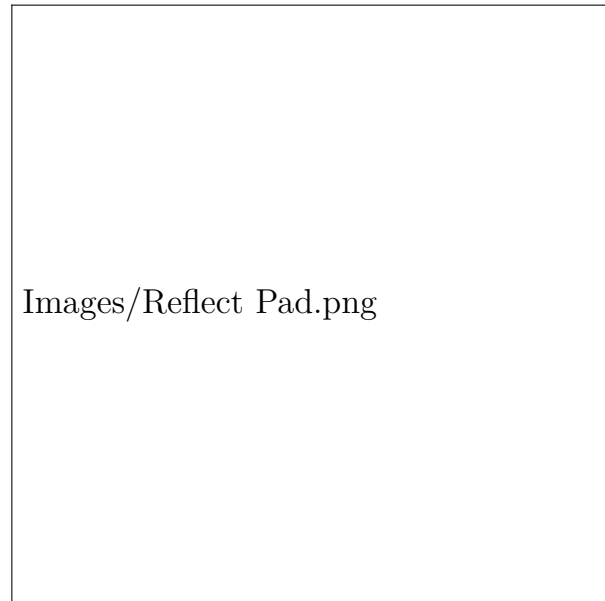


Figure 3.11 Reflection padding used in the generator network

Processing Element Architecture

The Processing Element (PE) is the basic arithmetic unit of the accelerator. Each PE computes one convolution result by multiplying input feature values with the corresponding kernel weights and accumulating the products.

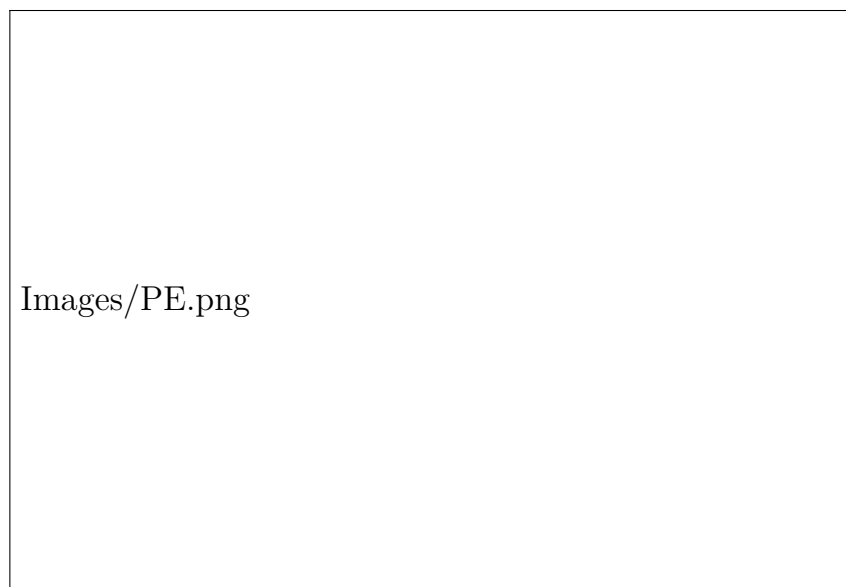


Figure 3.12 Processing Element (PE) architecture for convolution computation

[NXB3] Mau thuan MAC/cycle Fusion Core: o day ghi 16 PE x 8 = 128 MAC/cycle. Bang 4.x (throughput) ghi Fusion 'Peak MAC/cy = 256'. 128 vs 256

lech 2 lan. Lam ro: co the 256 dem ca mul+add (128 MAC = 256 OP). Thong nhat don vi MAC vs OP giua chuong 3 va 4. As shown in Figure 3.12, each PE contains eight parallel multipliers (eight MAC operations per cycle), an adder tree, an accumulation register, and bias addition. The multipliers are mapped to FPGA DSP resources, while the adder tree reduces the partial products with low latency. Multiple PEs are instantiated to form a PE cluster: the Fusion Core uses sixteen PEs ($PE_{SF} = 16$), giving $16 \times 8 = 128$ MAC operations per cycle when the inner MAC loop runs at $II = 1$. Each PE is assigned to a different output column or lane, which increases throughput by computing several convolution outputs in parallel while reusing the same buffered input data across lanes.

A key difficulty with FP16 accumulation is that a half-precision adder has a latency of three to four cycles, which would normally prevent a new accumulation every cycle. To still reach $II = 1$, the design uses a *rotating accumulator*: instead of a single accumulator, each PE keeps a small array of partial sums $psum[PE][D]$ and rotates the write index every cycle (for example $acc_idx = i \& 3$ for depth four). The distance between two read-after-write dependencies on the same slot then equals the accumulator depth, which is greater than or equal to the adder latency, so the pipeline never stalls. After the loop finishes, the partial sums in each slot are reduced into the final result.

Two-Pass Convolution-and-Normalization Datapath

The line buffers and PE cluster described above are combined into two reusable functional units: a **Convolution Functional Unit (Conv FU)** that contains the MAC array (the PE cluster), and a **Normalization/Activation Functional Unit (Norm/Act FU)** that performs channel normalization and the activation function. Because each convolution layer is followed by normalization and activation, the datapath is naturally organized as two passes over the feature map: a convolution pass produced by the Conv FU, followed by a normalization-and-activation pass produced by the Norm/Act FU. Figure 3.13 shows this datapath for a residual block, including input-feature buffering, weight buffering, the PE cluster, output buffering, the normalization/activation logic, and a skip-connection buffer.

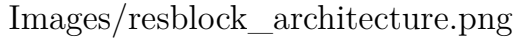
The image is a placeholder for a diagram showing the datapath of a residual block in the Fusion Core. It is intended to illustrate the sequence of operations: convolution followed by normalization.

Figure 3.13 Convolution-then-normalization datapath of the Fusion Core, shown for a residual block

Figure 3.14 illustrate how the convolution pass and the normalization/activation pass are overlapped. While the Norm/Act FU processes one row, the Conv FU can already compute the next row, so after an initial warm-up the two passes run concurrently. This overlap, together with the rotating accumulator that keeps the MAC loop at $II = 1$, is the main reason the design sustains a high MAC utilization.

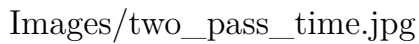
The image is a placeholder for a diagram showing the temporal overlap between the convolution and normalization/activation passes. It likely uses a timeline or state transition diagram to show how these two operations can be interleaved to improve hardware utilization.

Figure 3.14 Overlap of the convolution pass and the normalization/activation pass

Fusion Core Block Diagram

A single shared engine implements the initial convolution block, the nine residual blocks, and the global skip addition. Because the CBI, the nine residual blocks, and the global add all operate on the same $16 \times 16 \times 960$ tensor, they are mapped to one hardware instance that is invoked repeatedly with different

parameters, rather than replicated. To keep this as a single instance, every call passes the same set of ports; if some calls passed null pointers and others did not, the tool would infer two different instances and exceed the on-chip memory budget. The engine uses sixteen PEs with the rotating accumulator, and stores a large skip tensor in UltraRAM (URAM): a *skip-connection buffer* that holds the input of each residual block until its residual add and the CBI output until the global add. The complete block diagram, which brings together the AXI master interfaces, the input/weight/parameter buffers, the PE cluster, the normalization/activation logic, and the URAM skip buffers, is shown in Figure 3.15.

Images/FusionCore_HighLevel.png

Figure 3.15 Block diagram of the Fusion Core

3.4.3 UpConv Block Core

The UpConv Block Core performs the four up-sampling blocks of the generator. As shown in Figure 3.16, each block follows the same network pattern: a 3×3 transposed convolution, followed by channel normalization and a ReLU activation. Every transposed convolution uses stride two, so each block doubles the spatial resolution and halves the channel count, as summarized in Table 3.3.

Images/UCB_Model_architerture.png

Figure 3.16 Network pattern of an up-convolution block (UCB): a 3×3 transposed convolution followed by normalization and ReLU

Table 3.3 Resolution and channel changes through the four up-convolution blocks

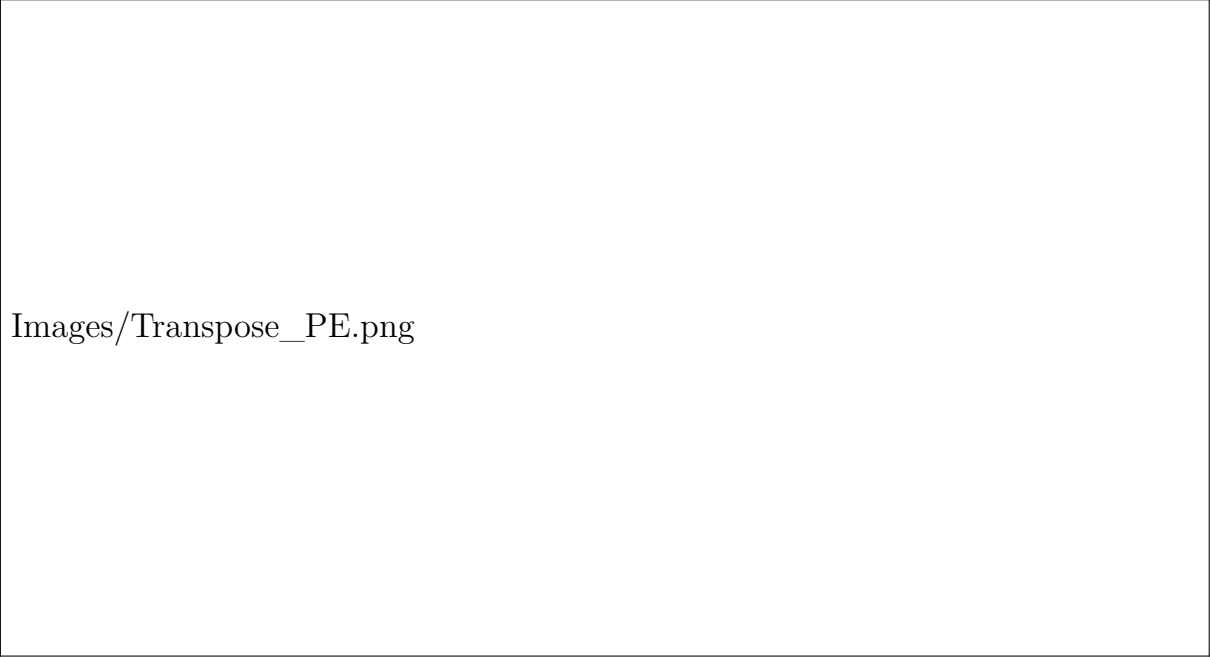
Block	Input	Output
UCB_0	$16 \times 16 \times 960$	$32 \times 32 \times 480$
UCB_1	$32 \times 32 \times 480$	$64 \times 64 \times 240$
UCB_2	$64 \times 64 \times 240$	$128 \times 128 \times 120$
UCB_3	$128 \times 128 \times 120$	$256 \times 256 \times 60$

Computation Model: Transposed Convolution as Scatter

Following the scatter formulation of transposed convolution introduced in Chapter 2, this core implements each up-convolution block as a *scatter-and-accumulate* operation: every input pixel distributes its weighted contributions to several output positions according to the 3×3 kernel and the stride-two pattern. Because of the stride, the input-to-output mapping is sparse, so for any given output row only part of the kernel taps actually contribute while the rest are skipped. As in the Fusion Core, a single shared engine serves all four blocks; the input source is selected by an internal mode parameter so that the tool infers only one hardware instance.

Hardware Architecture: Differences from the Fusion Core

At the block level, the UpConv Block Core reuses almost the same organization as the Fusion Core: the same AXI master interfaces, input/weight/parameter buffering, streaming line buffers, and the overall convolution-then-normalization datapath. The one essential difference is the Processing Element: the standard PE of the Fusion Core is replaced by a **Transpose PE**, shown in Figure 3.17. The Transpose PE keeps the same multiplier array, adder tree, and bias addition, but adds two extra stages at its output – a **Scatter** stage and an **Accumulator** stage.



Images/Transpose_PE.png

Figure 3.17 Block diagram of the Transpose PE in the UpConv Block Core

To understand why these two stages are needed, it helps to contrast the two operations. In the Fusion Core, the convolution is a *gather*: each output pixel collects a fixed 3×3 input window, so once a PE finishes its multiply-accumulate the result is already the final output value and can be written out in place. The computation is output-stationary, and a standard PE therefore needs nothing beyond its MAC and bias stages. The transposed convolution is the opposite, a *scatter*: one input pixel, multiplied by the kernel, contributes to several different output positions, and conversely each output position receives contributions from several input pixels and kernel taps that are produced in different loop iterations. A Transpose PE result is thus only a *partial* contribution to an output pixel, never its final value. This is precisely the gap that the Scatter and Accumulator stages close.

Why the Scatter stage. A normal PE always writes its result to the same, in-place output location. A Transpose PE cannot, because the upsampling pattern sends each partial result to a different, non-contiguous output position. The Scatter stage performs this routing: from the input pixel position and the active kernel tap it computes the destination address in the output feature map and steers the partial result there. For the stride-two transposed convolution used here, an input pixel at column w_i contributes through kernel tap k_w to output column $w_o = 2w_i + k_w - 1$; the Scatter stage applies this index mapping so that each partial result lands at the correct upsampled position. In short, it replaces the fixed write address of a

normal PE with a computed, data-dependent one dictated by the stride.

Why the Accumulator stage. Because several partial results are scattered onto the *same* output position at different times, that position cannot simply be overwritten – each new contribution has to be *added* to what is already there. The Accumulator stage carries out this read–modify–write: it reads the partial sum currently stored at the scattered output position, adds the new contribution, and writes the updated value back. These running sums are kept in a row-wide accumulation buffer (mapped to on-chip URAM) that persists across loop iterations until every input pixel has scattered its contribution; only at that point is an output pixel complete and ready for normalization. The Fusion Core needs neither stage, because its gather computation writes each output pixel exactly once and never has to read it back.

Execution Timing

[NXD7] Kiem chung timing UCB_0 : $tile_{41 \times 69,483} = 2,848,803$ (dung). Congreset 1,026 + load_params 162 + tile 2,848,803 + norm 23,363 = 2,873,354, NHUNG van banghi total 2,878,197. Lech convolution engine, in which the loading of input rows is overlapped with the computation of output rows. 5 convolution block, UCB_0 (input $16 \times 16 \times 960$, output $32 \times 32 \times 480$): after a short initialization (RESET_ROW_ACC, 1,026 cycles, and LOAD_PARAMS for bias, beta, and gamma, 162 cycles), the engine iterates the tile loop 41 times, each iteration loading a weight tile (4,449 cycles) and performing the 8-channel MAC compute (69,483 cycles). The tile loop therefore dominates at $41 \times 69,483 = 2,848,803$ cycles, followed by the normalization-and-write stage (statistics and normalization, 23,363 cycles), giving a total latency of 2,878,197 cycles for UCB_0 .

Images/timing_UCB.jpg

Figure 3.18 Timing path of the up-convolution engine for the first up-convolution block (UCB_0)

3.4.4 Conv77 Core

The Conv77 Core implements the final output block: reflection padding, a 7×7 convolution that maps the 60 feature channels to the 3 RGB channels, and a clipping operation. Its Processing Element reuses the same architecture as the Fusion Core PE – the same multiplier array, accumulation, and bias addition – and differs only in the adder tree. Because the kernel is 7×7 instead of 3×3 , each output value is the sum of a larger number of partial products, so the adder tree is widened and deepened accordingly to reduce them. Unlike the other two cores, the Conv77 Core uses only Block RAM (no URAM) for its small input-window and weight buffers. A final clipping to the range $[0, 1]$ is applied, as required by the HiFiC specification for normalized images.

3.4.5 On-Chip Memory Layout

Table 3.4 summarizes the main on-chip buffers. Large, persistent skip tensors and the up-convolution row accumulator are placed in URAM, while sliding-window buffers that require many parallel access ports are placed in Block RAM. For example, the Fusion input-window buffer is partitioned for sixteen parallel lanes; if it were placed in URAM it would require more blocks than the device provides, so it is mapped to BRAM instead.

Table 3.4 Main on-chip buffers of the generator accelerator

Buffer	Size	Type	Purpose
Skip-connection buffer	$16 \times 16 \times 960$	URAM	residual skip of each res_block
Global skip buffer	$16 \times 16 \times 960$	URAM	CBI output for the global add
Row accumulator	256×480	URAM (32 banks)	up-convolution output accumulation
Fusion window buffer	$4 \times W \times 60$	BRAM	sliding-window input feature map
Conv77 window buffer	$7 \times 14 \times 64$	BRAM	sliding window of the 7×7 convolution
Conv77 weight buffer	$3 \times 7 \times 7 \times 64$	BRAM	on-chip 7×7 weights

[NXD6] Bang on-chip buffers: 'Fusion window buffer = $4 \times W \times 60$ '. Fusion

Core xử lý tensor $16 \times 16 \times 960$ (960 kênh), không phải 60. Số '60' là số kênh đầu vào Conv77. Kiểm lại kích thước buffer Fusion, tránh nhầm với Conv77.

3.4.6 Top-Level Kernel Interface

The HLS kernel `full_generator_top` exposes a memory-mapped interface that allows the PS to configure and invoke the generator. Large tensors, weights, and parameters are mapped to AXI master ports, while scalar configuration values and the start/done handshake use an AXI4-Lite control interface. Table 3.5 lists the AXI master ports.

Table 3.5 AXI master ports of `full_generator_top`

Port	Direction	Contents
<code>gmem_x</code>	inout	input hyper-latent $16 \times 16 \times 960$
<code>gmem_wf</code> , <code>gmem_pf</code>	in	Fusion weights and parameters
<code>gmem_wu</code> , <code>gmem_pu</code>	in	UpConv weights and parameters
<code>gmem_y</code>	inout	UCB_3 output $256 \times 256 \times 60$ (read back by Conv77)
<code>gmem_wc</code> , <code>gmem_bc</code>	in	Conv77 weights and bias
<code>gmem_z</code>	out	final reconstructed image $256 \times 256 \times 3$

Assigning the input, weight, and output tensors to separate AXI bundles lets the tool generate independent memory adapters, so reads and writes can be scheduled with less port contention. The PS writes the base addresses and tensor dimensions through AXI4-Lite, starts the kernel, waits for completion, and then reads back the reconstructed image from `gmem_z`.

3.5 High-Level Synthesis and FPGA Deployment

High-Level Synthesis is used to transform the C/C++ accelerator description into RTL hardware. This approach reduces design time compared with writing the complete datapath manually in Verilog or VHDL, while still allowing hardware-specific optimization through HLS directives.

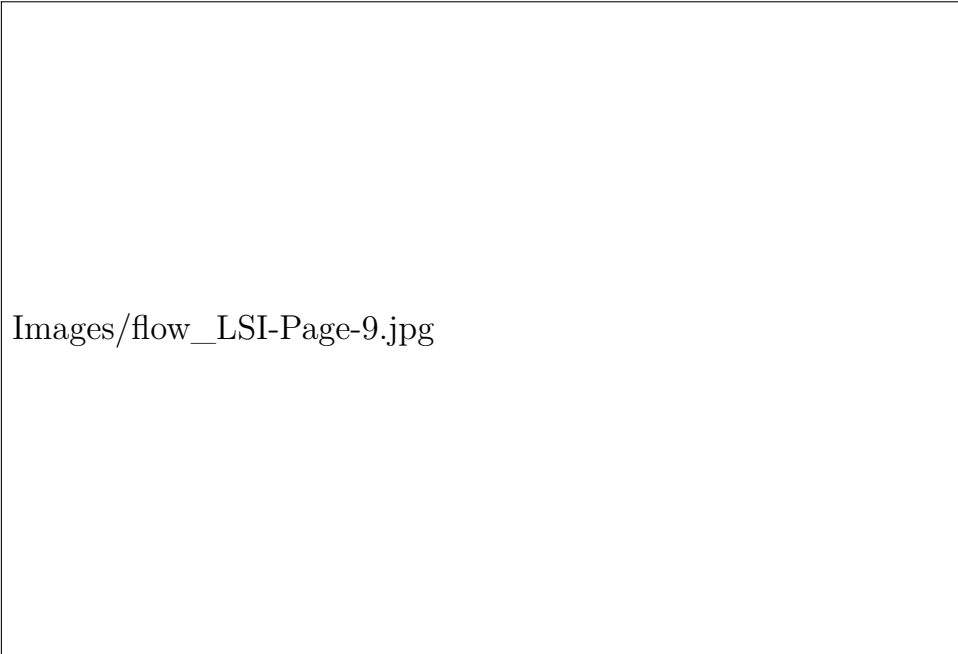


Figure 3.19 High-Level Synthesis design flow [?]

The deployment flow in Figure 3.19 consists of C/C++ design, functional simulation, synthesis, RTL export, Vivado integration, and board-level execution. During C/C++ simulation, the accelerator output is compared with the software reference to verify functional correctness. After synthesis, Vitis HLS reports latency estimates, initiation intervals, and resource usage, which are used to refine the architecture.

Several HLS techniques are applied in the implementation:

- **Loop pipelining:** overlaps loop iterations so that new data can enter the datapath every cycle or after a short initiation interval.
- **Array partitioning:** increases memory bandwidth for frequently accessed buffers such as windows, weights, and partial sums.
- **Dataflow scheduling:** enables loading, computation, and output writing functions to execute concurrently.
- **Interface directives:** map large data arrays to AXI master ports and scalar control values to AXI4-Lite registers.

After the accelerator is exported as an IP core, Vivado is used to connect it with the Zynq Processing System, AXI interconnects, DDR memory, clocking, and reset logic. The final bitstream configures the PL with the generator accelerator. On the software side, a Vitis application running on the ARM processor prepares

input buffers, writes configuration registers, launches the accelerator, and reads back the reconstructed image.

This implementation flow provides a complete path from neural network analysis to executable FPGA deployment. It also separates architectural design decisions in this chapter from the resource, timing, and performance evaluation reported in the following chapters.

3.6 Chapter Conclusion

This chapter presented the implementation architecture of the HiFiC GAN accelerator. The system was organized as a hardware–software co-design in which the PS handles control and flexible processing, while the PL implements the complete generator – the Fusion, UpConv, and Conv77 stages – as a single accelerator IP.

The chapter also described the internal blocks, including line buffers, sliding windows, reflection padding, the processing element with its rotating accumulator, the two convolution and normalization/activation functional units, the three generator cores, the on-chip buffer layout, and the AXI kernel interface. Finally, the HLS and FPGA deployment flow was introduced to show how the C/C++ design is synthesized, integrated, and controlled on the ZCU104 platform. The next chapter evaluates the resulting FPGA implementation in terms of resource utilization, system integration, and performance.

CHAPTER 4. EXPERIMENTAL RESULTS

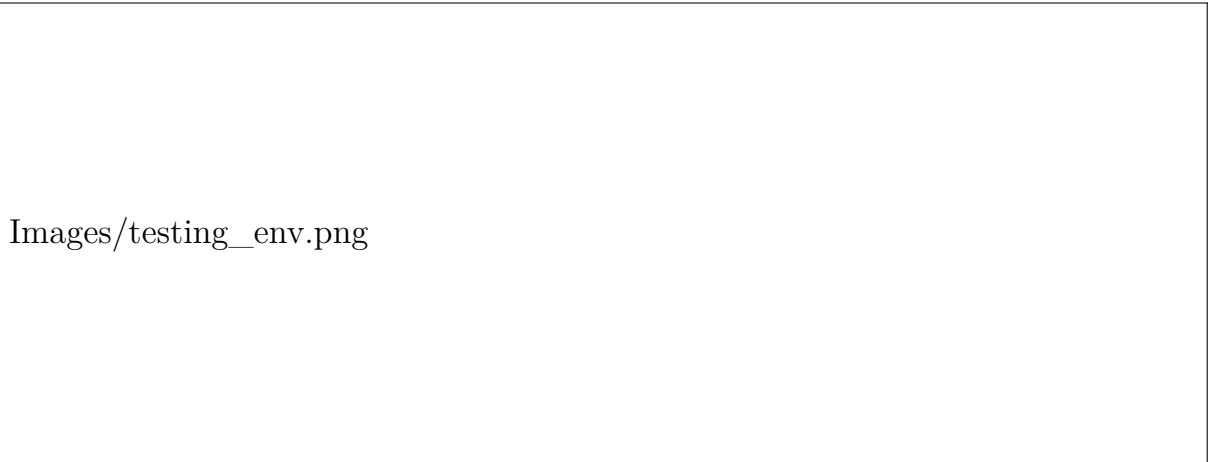
Chapter 4 presents the experimental procedure and evaluation results of the proposed quantized HiFiC GAN accelerator. Following an evaluation-oriented structure, this chapter first describes the accuracy evaluation environment and the quantization results, then reports FPGA resource utilization, analyzes performance, compares the hardware-oriented implementation with the software reference, and finally summarizes the main observations.

4.1 Accuracy Evaluation

The first objective of the evaluation is to verify that the hardware-oriented implementation preserves the functional behavior of the HiFiC inference pipeline. Since the proposed accelerator implements the complete generator, the verification process compares the HLS/FPGA-oriented output with the Python reference model using the same input tensors and model parameters.

4.1.1 Verification Environment

The overall testing methodology is shown in Figure 4.1. A set of test images (256×256 RGB) drives two paths in parallel: the proposed HiFiC GAN accelerator running on the ZCU104 FPGA (the design under test) and a reference model, namely the original HiFiC FP32 network implemented in Python/PyTorch. For each test item, the accelerator output is compared against the reference output to produce a PASS/FAIL verdict for hardware behavior verification, while the reconstructed images are compared to quantify quality and performance through metrics such as speedup, PSNR/SSIM, latency, and compression ratio.



Images/testing_env.png

Figure 4.1 Testing environment: the accelerator under test compared against the HiFiC FP32 reference model

Figure 4.2 illustrates the verification and deployment environment used in this thesis. The system follows a hardware–software co-design approach on the ZCU104 platform. The Processing System (PS) is responsible for software control, memory management, and accelerator invocation, while the Programmable Logic (PL) implements the custom generator accelerator.

Images/System Architecture.png

Figure 4.2 Verification and deployment environment on the ZCU104 platform

The software reference is executed in Python to generate golden outputs for comparison. The same input tensors, weights, and normalization parameters are then used by the HLS C/C++ implementation. During simulation, the output of the HLS model is compared with the Python reference to verify numerical consis-

tency. After successful HLS verification, the generated IP core is integrated into the Vivado block design shown in Figure 4.3.



Figure 4.3 Vivado block design of the proposed accelerator system

The target board used for implementation is the Zynq™ UltraScale+™ MP-SoC ZCU104, shown in Figure 4.4. The most relevant components for this thesis are summarized in Table 4.1. These components provide the processing system, programmable logic, external memory, SD-card boot interface, and display/output connections required for testing the accelerator.

Table 4.1 Relevant ZCU104 components for accelerator evaluation

[N~~XE~~1] Bang ZCU104 components danh so 1,3,5,6,7,27 (khong lien tuc). Neu la nhan theo so do board thi ghi chu '(numbering follows the board diagram in [ref])'; neu khong se kho hieu.

No.	Component
1	Zynq UltraScale+ XCZU7EV MPSoC
3	PL-side SODIMM DDR4 memory
5	USB 3.0 transceiver and USB 2.0 ULPI PHY
6	SD-card interface connector
7	Programmable Logic JTAG
27	DisplayPort connector



Images/ZCU.png

Figure 4.4 Zynq™ UltraScale+™ MPSoC ZCU104 evaluation board [?]

The verification flow consists of four main steps. First, the Python reference model produces the expected reconstructed output or intermediate feature output. Second, the HLS C/C++ model processes the same input data using the hardware-oriented implementation. Third, the two outputs are compared using objective image-quality metrics. Finally, after synthesis and implementation, the accelerator is controlled from Vitis on the ZCU104 platform to measure board-level behavior.

4.1.2 Functional Verification Results

Before board-level deployment, the hardware-oriented C/C++ model of every generator stage was validated against the FP32 golden tensors exported from the original HiFiC model through C-simulation (CSIM). A block is considered to PASS when its maximum absolute error stays below the tolerance assigned to that stage and no output element is reported as a mismatch. Table 4.2 summarizes the block-level results obtained on real input data.

Table 4.2 Block-level functional verification (CSIM) against the FP32 golden reference

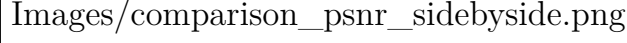
Block	Tensor shape (in $\rightarrow$ out)	Max abs. error	RMSE	Mismatch
CBI	$16 \times 16 \times 220 \rightarrow 960$	0.01953	0.00169	0 / 245,760
ResBlock	$16 \times 16 \times 960 \rightarrow 960$	0.01953	0.000978	0 / 245,760
GlobalAdd	$16 \times 16 \times 960$	0.02344	0.00178	0 / 245,760
UCB_0	$16 \times 16 \times 960 \rightarrow 32 \times 32 \times 480$	0.00781	0.000591	0 / 491,520
UCB_1	$32 \times 32 \times 480 \rightarrow 64 \times 64 \times 240$	0.01367	0.001061	0 / 983,040
UCB_2	$64 \times 64 \times 240 \rightarrow 128 \times 128 \times 120$	0.02148	0.002284	0 / 1,966,080
UCB_3	$128 \times 128 \times 120 \rightarrow 256 \times 256 \times 60$	0.01563	0.001405	0 / 3,932,160
Conv77	$256 \times 256 \times 60 \rightarrow 3$	0.488	0.162	0 / 196,608

Every block passes with zero element mismatches, which confirms that the mixed-precision FP16 datapath reproduces the reference behavior of the generator. For all feature-map stages the error magnitude is small, with an RMSE on the order of 10^{-3} . The Conv77 output is evaluated with a relaxed tolerance because its final Clip(0,1) together with the FP16 accumulation over a 7×7 window produces a slightly larger but still visually negligible deviation, which is further confirmed by the image-quality metrics reported in the following subsections.

4.1.3 Test Images and Evaluation Metrics

The evaluation uses representative RGB images resized to 256×256 , consistent with the input size used throughout this thesis. The reconstructed images are evaluated using image-quality metrics introduced in Chapter 2, including Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), and Mean Squared Error (MSE). These metrics are suitable for checking whether the quantized hardware-oriented implementation remains close to the software reference.

Figure 4.5 shows the PSNR comparison between the Python reference model and the HLS simulation model over 100 test images. The similarity of the two curves indicates that the HLS implementation follows the numerical behavior of the reference model closely enough for FPGA deployment.



Images/comparison_psnr_sidebyside.png

Figure 4.5 PSNR comparison between the Python reference model and the quantized HLS simulation model

Overall, the accuracy evaluation confirms that the hardware-oriented Res-Block implementation preserves the main reconstruction behavior of the HiFiC generator. Small numerical differences are expected because the accelerator uses a mixed-precision data path, but the observed quality trend remains close to the Python reference.

4.1.4 Mixed-Precision Quantization Results

This subsection evaluates the impact of the mixed-precision quantization described in Chapter 3, i.e., the transition from full-precision FP32 computation to a mixed-precision FP16 implementation, on compression efficiency and reconstructed image quality. The experiment uses a representative test image with a resolution of 256×256 . Two configurations are compared under identical conditions: the baseline FP32 model and the quantized mixed-precision FP16 model, where internal computations operate in FP16 while input and output interfaces remain in FP32. Table 4.3 summarizes the quantitative compression metrics obtained from both models.

Table 4.3 Comparison between FP32 and FP16 mixed-precision HiFiC models

Metric	FP32	FP16 Mixed	Delta
Original Size (bytes)	25,879	25,879	0
Bitstream Size (bytes)	6,522	6,517	-5
Output Image Size (bytes)	97,596	97,496	-100
Compression Ratio	3.97×	3.97×	0.00
Space Saving (%)	74.80	74.82	+0.02
Bits Per Pixel (bpp)	0.7961	0.7955	-0.0006
PSNR (dB)	19.08	19.05	-0.03
SSIM	0.8315	0.8314	-0.0001
MSE	804.315755	808.560496	4.244

[NXF4] PSNR = 19.08 dB rat thap so voi HiFiC goc (28.5 dB Bang 1.1, literature 28-32). Du delta FP32->FP16 nho (0.03 dB, hop ly), gia tri tuyet doi 19 dB can giai thich, va nen danh gia tren ≥ 3 anh thay vi 1 anh de tang do tin cay. The results indicate that the compression performance of the HiFiC model is preserved after quantization. The bitstream size is slightly reduced, while the compression ratio and space-saving metrics remain unchanged. This suggests that the probability distribution of the latent representation is not significantly affected by FP16 rounding errors, allowing the entropy coding process to operate efficiently. From a perceptual quality perspective, the degradation introduced by mixed-precision computation is negligible: the observed PSNR reduction of 0.03 dB is well below the threshold of perceptual visibility, and the SSIM metric remains nearly identical between the two configurations.

Figure 4.6 provides a visual comparison between the reconstructed images produced by the FP32 and FP16 models. No visible artifacts or structural distortions are observed in the quantized output, confirming that the proposed mixed-precision quantization strategy achieves a favorable balance between numerical efficiency and output fidelity.

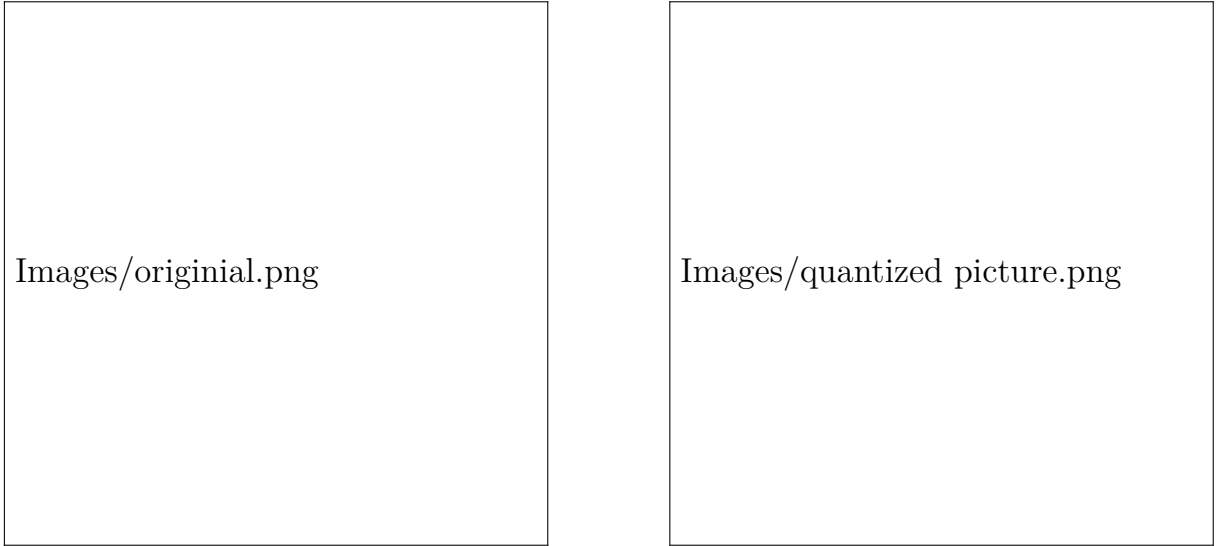


Figure 4.6 Visual comparison of reconstructed images: FP32 model (left) and FP16 mixed-precision model (right)

4.2 Resource Utilization and Power Measurement

After functional verification, the complete generator accelerator was synthesized and implemented on the ZCU104 FPGA platform to evaluate resource utilization. The implementation result reflects the cost of mapping all three generator stages (Fusion, UpConv, and Conv77), their local buffering, AXI-based data movement, and control logic into the Programmable Logic (PL). The target clock frequency is 300 MHz, and the implemented design closes timing with a maximum frequency of 308.74 MHz.

The hardware utilization report is summarized in Table 4.4. The implemented accelerator uses 205,233 LUTs, corresponding to 89% of the available LUT resources, and approximately 189,763 FFs, corresponding to 41% of the available flip-flop resources. For on-chip storage, the design consumes 510 BRAM 18Kb blocks, or 82% of the available BRAM resources, together with all 96 URAM blocks (100%). The arithmetic datapath uses 1,550 DSP slices, corresponding to 90% of the available DSP resources.

Table 4.4 Hardware resource utilization of the full HiFiC generator accelerator on the ZCU104

Resource	Full Generator Accelerator
LUTs	205,233 (89%)
FFs	189,763 (41%)
BRAMs (18Kb Block)	510 (82%)
URAMs	96 (100%)
DSPs	1,550 (90%)
Max frequency (MHz)	308.74

The utilization results show that the proposed accelerator is both computation-intensive and memory-intensive. DSP utilization is high because convolution dominates the generator workload and requires a large number of multiply-accumulate operations. BRAM and URAM utilization are also high because intermediate feature maps, line buffers, weights, the up-convolution row accumulator, and the skip-connection buffers must be stored locally to reduce external DDR access and sustain the pipelined datapath; in particular, all available URAM blocks are used.

To show how each core contributes to the overall cost, Table ?? reports the per-core resource utilization (absolute count and the corresponding share of the device resources). The Fusion and UpConv cores dominate DSP usage because they perform the bulk of the multiply-accumulate work, whereas the Conv77 core is comparatively LUT-heavy due to its final Clip(0,1) stage. The URAM blocks are concentrated in the UpConv core, which stores the row accumulator and the sliding-window buffers required by the transposed convolution.

In Table ??, the percentage in each cell is taken relative to the resources available on the XCZU7EV device (624 BRAM_18K, 1,728 DSP, 460,800 FF, 230,400 LUT, and 96 URAM). The figures correspond to the synthesis estimate of each core before the final data-buffer rebalancing step (PIPO double-buffering), which redistributes storage between BRAM and URAM and moves part of the FP16 multipliers from the DSP slices to LUT fabric without changing functionality. The fully implemented design therefore closes at the slightly different resource figures reported in Table 4.4 (1,550 DSPs, 510 BRAMs, all 96 URAMs, and 205,233 LUTs).

[NXF5] Mục 4.x tiêu đề 'Resource Utilization and Power Measurement' nhưng Power CHUA có số đo thực (đong 250 chỉ nói 'evaluated as part of board-level deployment', liệt kê dinh tinh). Yêu cầu: mức hóa đo công suất phải có W và

GOPS/W thuc, hoac doi tieu de bo 'Power Measurement', hoac bo sung Vivado power report (W) + tinh GOPS/W = 122.8 / P. Power measurement is evaluated as part of board-level deployment. In this design, the main contributors to dynamic power are expected to be DSP activity, BRAM/URAM access, AXI memory transfers, and clocked control logic. Therefore, reducing unnecessary memory movement and improving data reuse are important not only for performance but also for power efficiency.

4.3 Performance Evaluation

In addition to resource utilization, several performance metrics are used to evaluate the effectiveness of the proposed accelerator. These metrics include operating frequency, throughput, accelerated block, numerical precision, and speedup compared with the software-only baseline.

Table 4.5 summarizes the main performance results. The accelerator operates at 300 MHz and implements the complete HiFiC generator. The total compute volume of the generator is 90.34 GOP (counting the operations actually executed by the MAC engines), and the design processes one 256×256 image in 735.6 ms, giving a sustained throughput of 122.8 GOPS. The latency is dominated by the Fusion Core (568.8 ms), followed by the UpConv Core (149.0 ms) and the Conv77 Core (17.75 ms).

Table 4.5 Performance evaluation of the proposed accelerator

Metric	Result
FPGA platform	ZCU104 (XCZU7EV)
Operating frequency	300 MHz (Fmax 308.74 MHz)
Accelerated network	Full HiFiC generator
Precision	Mixed FP16
Compute volume	90.34 GOP
Total latency (per image)	735.6 ms
Throughput	122.8 GOPS

The total workload of the generator and its distribution across blocks are summarized in Table 4.6. The MAC count is derived directly from the tensor dimensions, using

$$\text{MACS}_{\text{conv}} = H_{\text{out}} W_{\text{out}} C_{\text{out}} C_{\text{in}} K_H K_W, \quad \text{MACS}_{\text{convT}} = H_{\text{in}} W_{\text{in}} C_{\text{in}} C_{\text{out}} K_H K_W,$$

for standard and transposed convolution, respectively. The nine residual blocks dominate the workload with 38.22 GMAC, whereas the four up-convolution blocks

each contribute an identical 1.062 GMAC: although every UCB halves the channel count, it quadruples the spatial resolution, so the transposed-convolution MAC count remains constant across the four stages.

Table 4.6 Compute volume (MAC operations) of the generator blocks

Block	$C_{in} \rightarrow C_{out}$	Spatial out	Kernel	MACs
CBI	$220 \rightarrow 960$	16×16	3×3	2.123 G
ResBlock $\times 9$	$960 \rightarrow 960$	16×16	3×3	38.220 G
GlobalAdd	—	16×16	—	~ 0
UCB_0	$960 \rightarrow 480$	32×32	3×3	1.062 G
UCB_1	$480 \rightarrow 240$	64×64	3×3	1.062 G
UCB_2	$240 \rightarrow 120$	128×128	3×3	1.062 G
UCB_3	$120 \rightarrow 60$	256×256	3×3	1.062 G
Conv77	$60 \rightarrow 3$	256×256	7×7	0.578 G
Total				45.17 GMAC (90.34 GOP)

The latency of the design is deterministic and is obtained from the per-loop synthesis report multiplied by the actual trip counts. Table 4.7 lists the cycle count and the corresponding latency at 300 MHz for each stage. The Fusion Core accounts for the largest share (568.8 ms), driven by the nine residual blocks; the UpConv Core takes 149.0 ms after the dataflow optimization, and the Conv77 Core completes in about 17.75 ms.

Table 4.7 Latency breakdown of the generator stages at 300 MHz

Stage	Component	Cycles	Latency
Fusion	CBI	8,980,370	29.93 ms
	ResBlock $\times 9$	161,646,705	538.82 ms
	GlobalAdd	15,507	0.05 ms
UpConv	$4 \times$ UCB	—	149.0 ms
Conv77	Conv 7×7 + Clip	~ 5.3 M	17.75 ms
Total (per image)			735.6 ms

Table 4.8 Per-core throughput and MAC-array efficiency at 300 MHz

Core	Peak MAC/cy	Peak GOPS	Latency	Achieved GOPS	MAC eff.
Fusion	256	153.6	568.8 ms	141.9	92%
UpConv	128	76.8	149.0 ms	57.0	74%
Conv77	448	268.8	17.75 ms	65.1	24%
Full generator	—	—	735.6 ms	122.8	—

The performance result shows that the accelerator benefits from parallel DSP computation and local on-chip buffering. The Fusion Core, which executes the convolution-heavy residual blocks, reaches about 92% of its peak MAC throughput thanks to the rotating accumulator and the overlapped convolution and normalization passes; the UpConv and Conv77 cores are less MAC-bound because their latency also includes pipeline fill/drain and boundary handling. The high BRAM, URAM, and DSP utilization indicates that further performance scaling must be balanced carefully against the remaining device resources.

4.4 Comparison

To the best of our knowledge, the HiFiC model has not previously been implemented on an FPGA, so there is no prior hardware accelerator of the same model to compare against directly. Therefore, this section evaluates the proposed accelerator by comparing its performance against the software execution baseline, focusing on the acceleration ratio (speedup) and throughput.

The software baseline corresponds to the full generator inference profiled on an Intel Core i9 CPU operating at 2.2 GHz, which takes 1073.19 s per image, as reported in the runtime breakdown of Table 3.2 in Chapter 3. This long CPU runtime is a direct consequence of the very large feature tensors of the generator (for example, $16 \times 16 \times 960$ inside each residual block). The proposed accelerator executes the same generator on the ZCU104 platform at 300 MHz in 735.6 ms using mixed FP16 precision. Table 4.9 summarizes the comparison.

Table 4.9 Performance comparison between the software baseline and the proposed FPGA accelerator

Platform	Device	Precision	Latency	Speedup
Software (CPU)	Intel Core i9 @ 2.2 GHz	FP32	1073.19 s	1×
Proposed FPGA	ZCU104 (XCZU7EV) @ 300 MHz	Mixed FP16	735.6 ms	$\approx 1459\times$

The comparison shows that running the full generator on the FPGA reduces the per-image processing time from 1073.19 s on the CPU to 735.6 ms, an acceleration of approximately $1459\times$, while sustaining a throughput of 122.8 GOPS. This large speedup is obtained even though the FPGA operates at a much lower clock frequency than the CPU, because the accelerator exploits spatial parallelism through many processing elements, local on-chip buffering, and overlapped convolution and normalization passes that keep the MAC datapath busy. The result confirms that dedicated hardware is essential for the compute-bound HiFiC generator, whose very large feature tensors make plain CPU execution impractical.

4.5 Chapter Conclusion

Chapter 4 presented the experimental results of the proposed quantized HiFiC GAN accelerator. Block-level C-simulation against the FP32 golden reference passed with zero element mismatches across all generator stages, and the accuracy evaluation further compared the HLS simulation output with the Python reference model using PSNR over 100 test images; the mixed-precision quantization results showed that compression ratio, PSNR, and SSIM are preserved when moving from FP32 to FP16. The resource utilization report showed that the design uses a large portion of the available DSP, BRAM, and URAM resources (all URAM blocks are used), which is expected for a highly parallel full-generator accelerator. The performance evaluation demonstrated an operating frequency of 300 MHz (timing closed at 308.74 MHz), a total compute volume of 90.34 GOP, a total latency of 735.6 ms per image (568.8 ms Fusion, 149.0 ms UpConv, 17.75 ms Conv77), a throughput of 122.8 GOPS, and an acceleration of approximately $1459\times$ over the CPU software baseline.

Overall, the results confirm that the proposed hardware–software co-design is suitable for accelerating the complete HiFiC generator while preserving reconstruction quality.

[NXG2] Heading dung \section*{CONCLUSION} nhưng addcontentsline ghi 'CHAPTER 5. CONCLUSION'. Chuong 1-4 deu 'CHAPTER n. TITLE' o heading. Thong nhat: doi heading thanh 'CHAPTER 5. CONCLUSION'.

CONCLUSION

Key Features

The hardware–software co-design of a quantized HiFiC GAN accelerator for learned image compression has yielded positive results, demonstrating the capability of adversarial learning in optimizing both visual representation and reconstruction. A standout achievement of this research is the successful transition from algorithmic theory to a high-performance hardware implementation. Experimental results indicate that the hardware-accelerated generator processes one image in 735.6 ms, about $1459\times$ faster than the CPU software execution (1073.19 s), while preserving the reconstruction quality of the original FP32 model after mixed-precision FP16 quantization.

The main features of the proposed design can be summarized as follows:

- **Hardware–software co-design:** the complete HiFiC generator (the Fusion, UpConv, and Conv77 stages) is offloaded to a single custom FPGA IP, while the compression front-end and model orchestration remain in software on the Processing System.
- **Mixed-precision quantization:** the model is quantized to FP16 internally with FP32 interfaces, reducing memory footprint and computation cost with negligible loss in PSNR and SSIM.
- **Efficient generator accelerator:** streaming line buffers, sliding-window generation, a parallel PE cluster with a rotating accumulator (sustaining $II = 1$ in FP16), and shared convolution and normalization/activation engines minimize off-chip memory traffic and exploit the regular convolution structure of the generator.
- **Practical deployment:** the accelerator is implemented and evaluated on the ZCU104 platform at 300 MHz (timing closed at 308.74 MHz), processing one image in 735.6 ms at a throughput of 122.8 GOPS, about $1459\times$ faster than the CPU software baseline.

The project effectively bridges the gap between deep learning complexity and practical efficiency, showcasing a robust and adaptable solution for next-generation image processing.

Future Works

Moving forward, the project will focus on several key trajectories to transition the proposed accelerator from a research prototype toward a robust, real-world solution:

[NXF6] Future Works ghi 'stabilizing the adversarial training process and mitigating mode collapse'. Do an chi lam INFERENCE (decoder), KHONG huan luyen GAN (PTQ, khong retrain). Muc nay lac de so voi pham vi inference-accelerator. Nen thay bang huong mo rong phan cung (do phan giai >256, pipelining giua core, do cong suat thuc, INT8).**Architectural refinement:** stabilizing the adversarial training process and mitigating issues such as mode collapse to ensure consistent output quality. **Real-time deployment:** further optimizing the model for embedded and FPGA platforms, specifically targeting reduced computational complexity and lower latency. **Scalability:** extending the design to handle ultra-high-resolution datasets and diverse visual domains, ensuring robustness across different content types. **Hybrid modeling:** exploring the integration of GANs with Vision Transformers or Diffusion Models to push the boundaries of reconstruction fidelity. **Secure and efficient transmission:** integrating enhanced security measures and optimized bit-rate allocation, transforming the model into a versatile tool for high-efficiency, secure image transmission in bandwidth-constrained environments.